PA-DSE: Feasibility-Aware Design Space Exploration for High-Level Synthesis via Hierarchical Evidence-Bounded Pruning

***Abstract*—High-Level Synthesis (HLS) design-space exploration is dominated by synthesis failures: much of the budget is spent on configurations the tool will reject. Classical optimizers treat failures as dominated samples; learned surrogates require offline training.**

We propose PA-DSE, a feasibility-aware DSE policy built on a single principle: action strength must not exceed evidence strength. PA-DSE combines a Static Constraint Filter (SCF) for tool-documented rules with a Dynamic Failure Risk Learner (DFRL) that, within a single run, hard-skips configurations matching learned failure signatures and reorders the remaining queue by online risk estimates.

On eight benchmarks across PandA-Bambu and Dynamatic, PA-DSE reaches 92.5% success rate on Bambu, compared with 85.9% for an AutoScaleDSE-style [1] random-forest classifier, and 91.3% on Dynamatic, compared with 90.2% for GP-BO [2]. It reduces wasted calls by $1.9\times$ and halves time-to-first-feasible relative to the strongest baselines. An 8-way ablation confirms the evidence hierarchy: giving soft risk scores hard-skip authority causes order-of-magnitude variance growth. PA-DSE requires no offline training, adds only ~0.25% runtime overhead, and remains *decision-traceable*: every skip is justified by a named rule or learned signature. Code: https://github.com/jane09250410/hls-dse.

***Index Terms*—High-Level Synthesis, Design Space Exploration, Feasibility-Aware Optimization, Interpretable Machine Learning, PandA-Bambu, Dynamatic.**

I. Introduction

HIGH-Level Synthesis (HLS) enables rapid generation of hardware accelerators from C/C++ specifications, but the resulting design space is large and irregular. A single benchmark kernel may expose dozens of synthesis pragmas, each controlling loop pipelining, memory partitioning, array-to-channel mapping, or resource binding. For two representative HLS tools, PandA-Bambu [3] and Dynamatic [4], the configurations we consider in this work number 420 and 192 respectively, and the space grows combinatorially with kernel complexity. Exhaustive exploration is impractical: a single synthesis run typically takes 2–10 seconds for small kernels and tens of seconds for larger ones, so brute-force evaluation of even a moderately sized space consumes hours of wall-clock time.

Design Space Exploration (DSE) methods attempt to navigate this space efficiently. Classical approaches draw on general-purpose optimization: simulated annealing, genetic algorithms, and Bayesian optimization with Gaussian Processes [5] have all been adapted to HLS DSE [2], [6], [7]. More recent work applies machine learning to *predict* synthesis outcomes before invoking the tool [1], [8]–[11], potentially reducing the number of expensive tool invocations. Both families, however, share a blind spot: they treat synthesis failure as just another low-objective sample, rather than as a signal carrying structural information about which regions of the design space are *systematically* infeasible.

Feasibility is the dominant cost driver. In our experiments on eight Bambu benchmarks, roughly one-third of configurations in the raw design space are provably infeasible due to documented parameter incompatibilities (e.g., MEM_ACC_11 memory controllers with multi-channel binding). A larger fraction (often more than half) are *likely* infeasible due to pragma-level interactions (loop pipelining combined with certain memory/channel choices) that the released Bambu v0.9.8 grid rejects across all eight benchmarks. Random sampling against this space achieves only 15% success rate (SR) at budget $B{=}60$ on Bambu. The problem is not that a good sampler cannot find feasible configurations; it is that most of the budget is wasted on configurations the tool will reject, often with error messages that are informative if anyone bothers to read them.

PA-DSE is a feasibility-aware DSE policy that learns from tool error messages online, during a single exploration run, without offline training or surrogate modeling. The policy is organized around a single design principle we call the *evidence hierarchy*: action strength must not exceed evidence strength.

PA-DSE has two layers, the second of which contains two complementary sub-components:

- **Layer 1 (SCF, Static Constraint Filter).** Configurations matching tool-documented incompatibility rules are permanently removed before exploration begins. This is the strongest action class—cross-run permanent block—and is reserved for proven constraints with zero false-positive risk. Configurations matching weaker source-level heuristics are instead labeled as *suppressed*: they remain in the queue with a lower-priority risk label and may be reordered or evaluated later under online evidence.
- **Layer 2 (DFRL, Dynamic Failure Risk Learning).** An online policy that accumulates empirical evidence during a single exploration run. It has two sub-components operating at different evidence strengths:
  - **RPE (Recurrent Pattern Extractor).** After repeated observations of the same error type, RPE extracts typed failure *signatures*—parameter conditions that consistently co-occur with that error type and discriminate failures from successes. Matching configurations are hard-skipped for the remainder of the run, subject to a small probe rate that keeps signatures

honest.

– **OFRS (Online Failure Risk Scorer).** A lightweight per-dimension failure-rate tracker that reorders the remaining queue each iteration by a smoothed risk score. OFRS never skips configurations; it only changes their evaluation order, since per-dimension marginal statistics are too noisy to justify permanent exclusion.

The two layers are complementary, not redundant, and their interplay is tool-dependent. On Bambu, SCF removes 33% of the space as provably infeasible; disabling it (DFRL-only) drops SR from 84.5% to 80.0% at B=30 (Table V), because DFRL would otherwise spend budget observing catastrophic configurations rather than promising ones. On Dynamatic, the tool exposes no documented incompatibility rules ($\mathcal{C}_{\text{blk}} = \emptyset$), so SCF is a no-op; DFRL alone drives 89.6% SR at B=30, confirming that DFRL is not parasitic on SCF. The two layers therefore operate at different scopes: SCF on tools that expose static constraints, DFRL on all tools.

Contributions. This paper makes four contributions.

1) **Policy design.** We propose PA-DSE, a two-layer feasibility-aware DSE policy organized around the *evidence hierarchy* principle. Layer 1 (SCF) handles static, tool-documented constraints; Layer 2 (DFRL) is online and contains two sub-components (RPE and OFRS) that operate at different evidence strengths. Each layer's action strength is explicitly justified by the evidence it has access to (§III).
2) **Empirical validation across tools.** We evaluate PA-DSE on two qualitatively different HLS tools (Bambu, static scheduling, C-centric; Dynamatic, elastic dataflow, LLVM-based) across eight shared benchmarks. PA-DSE achieves 92.5% SR on Bambu (vs. 85.9% for the best baseline) and 91.3% SR on the five Dynamatic benchmarks with non-trivial failure rates (vs. 90.2% for GP-BO), while halving time-to-first-feasible compared to the strongest baselines (§V). Using the same benchmarks on both tools reveals a structural asymmetry: Bambu failures are pragma-driven (source-independent), while Dynamatic failures are graph-complexity-driven (MILP difficulty scales with dataflow-graph complexity), motivating the two-layer design.
3) **Mechanism isolation.** An 8-way ablation shows that (a) OFRS reordering is the dominant contributor at small budgets (B=30–60), (b) SCF acts as a safety net preventing DFRL from learning from catastrophic configurations, and (c) at B=120, RPE activates and provides a +36.3 pp SR gain over OFRS-only by hard-skipping configurations matching a learned failure signature (§V-G, §V-I).
4) **Near-zero runtime overhead.** The algorithmic overhead of PA-DSE is 5.3 ms per iteration on Bambu and 2.0 ms per iteration on Dynamatic, against synthesis times of 2.1 s and 16.5 s respectively, yielding ratios of 1:403 and 1:8164 (§V-J).

Scope. PA-DSE is not a general-purpose synthesis prediction model. RPE signatures are per-run and do not persist across benchmarks, and the static rules are tool-specific. Its strength lies in on-the-fly, within-run adaptation with inspectable state. Limitations are discussed in §VI.

II. Background and Related Work

A. HLS Design Space and Feasibility

High-Level Synthesis compiles a C/C++ specification into RTL, subject to user-specified pragmas. The pragmas form a combinatorial *configuration space* $\mathcal{C}$: each configuration $c \in \mathcal{C}$ is a discrete point (choice of loop pipelining, memory controllers, channel count, clock period target, etc.). Given a synthesis tool $\mathcal{T}$, evaluating c means invoking $\mathcal{T}$ and observing an outcome $y \in \{\text{success}(q), \text{fail}(\text{type})\}$, where q is a quality-of-result vector (area, latency) and type is an error category extracted from the tool log.

Two tools anchor our evaluation. **PandA-Bambu** [3] is a source-to-source HLS compiler with a statically scheduled datapath. Its pragma surface includes memory controller type (`MEM_ACC_11`, `MEM_ACC_N1`, `MEM_ACC_NN`), channel count, loop pipelining, and binding strategy. **Dynamatic** [4] generates dynamically scheduled (elastic dataflow) circuits and exposes a different surface: buffer placement, slack, fast-token paths, and target clock period. The two tools differ substantially in scheduling philosophy and failure mode distribution, making them a useful pair for stressing tool-generality claims.

Feasibility as a first-class outcome. A large fraction of $\mathcal{C}$ produces outright synthesis failure rather than merely poor-quality success. On Bambu with our chosen pragma ranges, two rule-provable incompatibility patterns alone account for 33.3% of the 420-point space: `MEM_ACC_11` requires exactly one channel, and `MEM_ACC_NN` requires at least two, so their union with the full channel-count range produces a deterministic infeasibility band. Beyond this, the released Bambu v0.9.8 grid exposes an additional pragma-driven high-risk region: configurations with loop pipelining enabled fail across all eight benchmarks under our pragma ranges. A DSE method unaware of these structural failures will repeatedly consume budget on configurations the tool cannot synthesize.

B. Related Work

a) General-purpose optimizers for HLS DSE.: Simulated annealing, genetic algorithms, and Bayesian optimization are widely used as HLS DSE backends because they make minimal assumptions about the objective [2], [6], [7]. Their blind spot is that they treat synthesis failure as a dominated sample rather than as structural information. In our Bambu evaluation, simulated annealing reaches 18.4% SR and genetic algorithms reach 50.6% at B=60, with high variance that reflects their sensitivity to the fraction of infeasible configurations. Bayesian optimization with Gaussian processes [5] fares better (71.3%) because its surrogate implicitly learns the failure boundary, but the surrogate is slow to warm up on small budgets.

b) Learned surrogates and classifiers.: A more recent line of work trains machine-learning models to predict synthesis outcomes before calling the tool. Approaches range from per-benchmark regressors and reinforcement-learning agents [8] to graph neural networks operating on LLVM IR or

custom HLS-specific graphs [9], [12], [13] to random forests in a scalable surrogate framework [1], [7]. Hybrid pipelines that combine learned surrogates with Bayesian optimization have also been proposed [10], [11]. These methods can be effective, but they share two weaknesses. First, they require offline training, which in turn requires a labeled sample of the very design space they are meant to explore, often a chicken-and-egg problem for new tool versions or new benchmarks. Second, they produce point predictions whose failure modes are hard to interpret: when a surrogate confidently predicts success but the tool fails, debugging is difficult because the surrogate has no notion of *why* it failed. Our random-forest baseline (RF, 85.9% SR on Bambu) is a proxy for this family under our tool versions and budgets, not a full re-implementation of any specific prior system.

c) Meta-learning and cross-benchmark transfer.: Recent work has explored transfer and meta-learning to amortize DSE cost across benchmarks. These approaches are complementary to PA-DSE: they focus on *cross-benchmark* knowledge transfer, whereas PA-DSE operates *within a single run* and does not persist learned signatures across runs. A potential extension is to use PA-DSE-extracted signatures as meta-features for cross-run transfer.

d) Rule-based filtering.: Rule-based feasibility pruning has been explored in autotuning contexts outside of HLS. Closer to our setting, HGBO-DSE [10] prunes invalid configurations via a tree-structured modeler before Bayesian optimization. Our work differs in two respects: (i) PA-DSE's feasibility pruning operates at *two* evidence strengths (proven rules via SCF and online-learned signatures via RPE) rather than a single pre-search filter, and (ii) PA-DSE does not require offline surrogate training, trading some long-budget optimality for single-run operation.

e) Positioning.: PA-DSE occupies a different design point from all of the above: it is (i) training-free, (ii) single-run (no cross-run persistence of DFRL state), (iii) decision-traceable at every step (every skip is attributable to a rule or a signature), and (iv) near-zero-overhead (algorithmic cost $< 1\%$ of synthesis cost). Its goal is not to outperform all learned methods in the long-budget limit, but to provide a robust, inspectable DSE policy that works well at the small budgets typical of interactive development.

To our knowledge, PA-DSE is the first HLS DSE framework to make the coupling between action strength and evidence strength a first-class design principle, and to show empirically that violating this coupling (e.g., granting OFRS skip authority) causes catastrophic variance. Table I summarizes how PA-DSE relates to the closest prior work along key design axes.

TABLE I
POSITIONING OF PA-DSE RELATIVE TO THE CLOSEST PRIOR WORK. "ACTION–EVIDENCE COUPLING" REFERS TO WHETHER THE SYSTEM EXPLICITLY RESTRICTS EACH COMPONENT'S ACTION CLASS BASED ON THE STRENGTH OF ITS EVIDENCE SOURCE.

	PA-DSE	**HGBO -DSE**	**Auto-Scale**	**GP-BO**
Surrogate / model type	None	GNN	RF	GP
Offline training	No	Yes	Yes	No[†]
Feasibility filter	2-level	1-level	No	No
Online adaptation	DFRL	No	No	Online
Action–evidence coupling	Yes	No	No	No
Decision traceable	Yes	No	No	No

[†] GP-BO fits its Gaussian-process surrogate online within a single run; no separate offline training corpus is required.

III. PA-DSE FRAMEWORK

A. Problem Setting and Policy Design

We frame feasibility-aware DSE as an *online budgeted exploration problem* and describe PA-DSE as a *deterministic hierarchical exploration policy* for this problem. We do not claim that PA-DSE solves an optimization problem in a formal sense; rather, we use the problem framing to clarify what the policy is trying to achieve and what trade-offs it makes.

State. At step t, the observable state is $\mathcal{S}_t = (Q_t, \mathcal{H}_t, \mathcal{P}_t, \mathcal{W}_t)$, where Q_t is the remaining evaluation queue, $\mathcal{H}_t = \{(c_j, y_j)\}_{j<t}$ is the observation history with $y_j \in \{\text{success}, (\text{fail}, \text{type}_j)\}$, $\mathcal{P}_t$ is the current set of active RPE signatures, and $\mathcal{W}_t$ is the current OFRS statistics.

Actions. For each configuration $c \in Q_t$, the policy selects one of: EVALUATE (consume one budget unit, observe outcome), SKIP (permanently remove from Q_t), or DEFER (reorder within Q_t).

Operational goals. The policy aims to:

1) Maximize the number of feasible designs discovered within a fixed budget B.
2) Minimize false skips (configurations incorrectly removed that were actually feasible).
3) Secondarily, minimize time-to-first-feasible and maximize best-QoR-under-budget.

These are *evaluation criteria* used to assess policy quality, not formal constraints enforced by the algorithm. In particular, the false skip rate is an *empirical safety target*, not an online guarantee: PA-DSE does not have a mechanism that provably bounds false skips below any threshold. Instead, it relies on the evidence hierarchy (Section III-B) and conservative activation rules to keep false skips low in practice.

Policy structure. PA-DSE implements a deterministic, hierarchical policy with *three action classes*, each gated by a qualitatively distinct evidence type:

1) **Permanent block** (cross-run, on *proven* evidence): if c matches a sound blocking rule in $\mathcal{C}_{\text{blk}}$, SKIP. The skip persists across runs because the underlying evidence (tool documentation) does not depend on runtime observations.
2) **Hard skip** (run-scoped, on *typed recurrent empirical* evidence): if c matches an active RPE signature with $\text{conf}(\pi) \geq \theta$, SKIP. The skip is scoped to the current run because the underlying evidence is observational and may not generalize.
3) **Reorder only** (within-run, on *marginal empirical* evidence): otherwise, rank c by the OFRS risk score $r(c)$ and DEFER high-risk configurations to the tail of the queue. No configuration is excluded.

The three action classes are ordered by strength: permanent block $\succ$ hard skip $\succ$ reorder. The evidence types backing

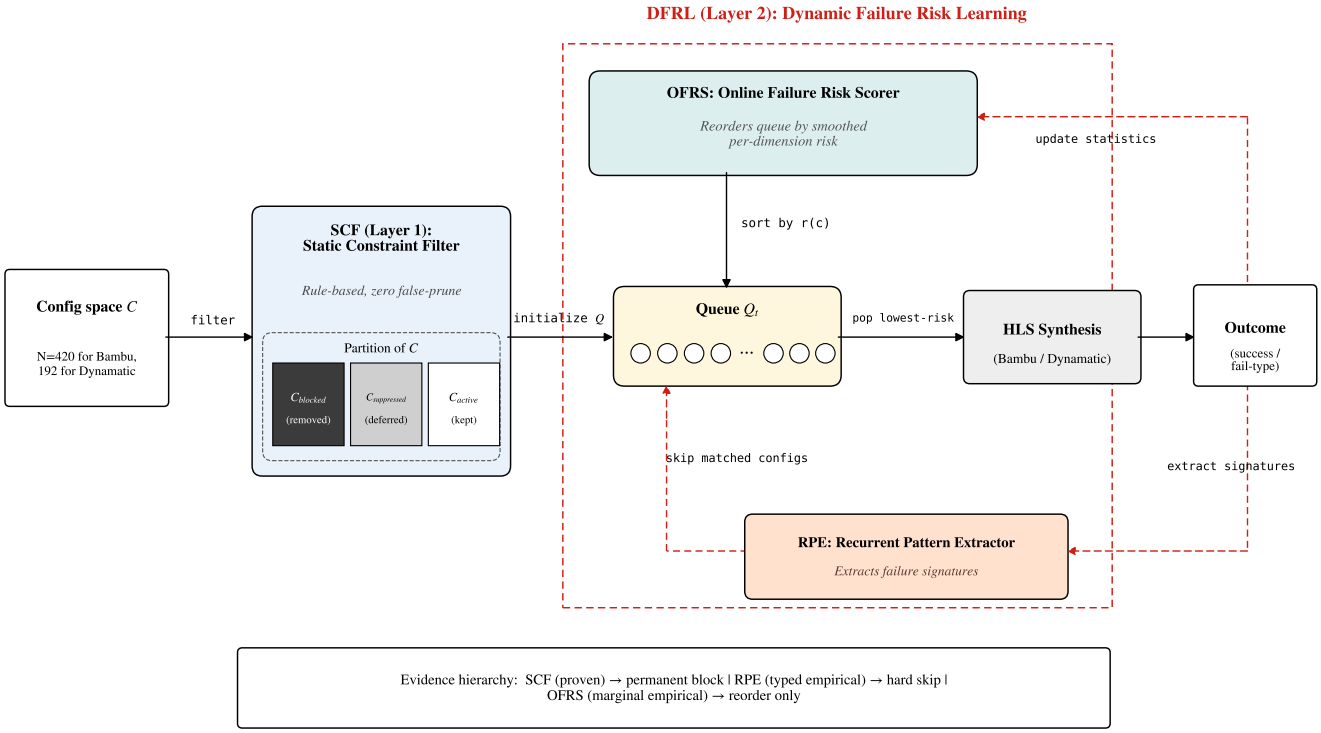

Fig. 1. PA-DSE architecture. Layer 1 (SCF) performs offline rule-based filtering and partitions the config space $\mathcal{C}$ into *blocked*, *suppressed*, and *active* subsets. Layer 2 (DFRL) operates online: OFRS reorders the remaining queue each iteration using smoothed per-dimension failure statistics; RPE extracts typed failure signatures from observed errors and removes matching configurations from the queue for the remainder of the run. Dashed red arrows denote the online feedback loop. Each component's action class (permanent block, hard skip, reorder) is strictly bounded by its evidence strength (bottom caption).

TABLE II
PA-DSE EVIDENCE HIERARCHY

Evidence Source	Strength	Action	Scope
Tool-documented constraint	Proven	Permanent block	All runs
Source-code heuristic	Medium (empirical)	Queue ordering prior	All runs
Recurrent failure signature	High (empirical)	Hard skip	Current run
Per-dimension failure stats	Low–medium	Queue reordering	Current run

them are ordered by certainty: proven $\succ$ typed empirical $\succ$ marginal empirical. The pairing is one-to-one and the two orderings align—this is the *evidence hierarchy* formalized in Section III-B.

B. Evidence Hierarchy

PA-DSE is organized around a single design principle: *action strength must not exceed evidence strength*. Table II summarizes the mapping. Only proven constraints (tool documentation) trigger permanent, cross-run blocking. RPE signatures trigger run-scoped hard skip based on empirical evidence accumulated during exploration. OFRS influences evaluation order but never permanently excludes any configuration.

Why three tiers. SCF and OFRS together cover only the extremes of the evidence spectrum. SCF requires proven evidence (tool documentation), which is available only for failure modes the tool vendor has explicitly characterized; the Bambu pipeline-conflict region (§V-I) is empirically recurrent but undocumented, and SCF cannot cover it without violating its zero-false-positive contract. OFRS sits at the other end and works from marginal per-dimension statistics that mix across failure types; granting it skip authority produces an order-of-magnitude variance increase (§V-G, SCF+OFRS-as-skip). RPE sits between them: its evidence is typed and concentrated within a run, which is stronger than OFRS's marginal statistics but weaker than tool documentation, so it supports run-scoped hard skip but not cross-run permanent block. Collapsing this middle tier degrades the policy in one of two ways. Downgrading RPE's action to reorder makes it redundant with OFRS (§V-G, RPE-as-reorder). Upgrading marginal statistics to skip authority breaks the evidence–action coupling. The three-tier structure is therefore the minimal configuration that covers the evidence types we encounter in HLS DSE.

Budget-conditional activation. Not every tier is active at every budget. In the full PA-DSE configuration at small budgets (B=30–60), OFRS reordering surfaces feasible configurations fast enough that τ same-type failures rarely accumulate in the queue's reachable prefix; RPE therefore remains dormant within this configuration, and PA-DSE is effectively driven by SCF + OFRS. (When OFRS is removed—the SCF+RPE ablation in §V-G—RPE does activate and reaches 57.8% SR at B=30, demonstrating that the dormancy is a property of the full configuration, not of RPE in isolation.) At extended budgets (B=120 on Bambu, §V-I), the queue extends into higher-failure-rate regions where type-specific

failure evidence concentrates, RPE activates within the full configuration, and the middle tier becomes load-bearing. Each tier engages when its activation condition is met by the evidence stream, and the system adapts to the budget regime without user intervention.

C. Failure Taxonomy (Modeling Assumption)

Assumption. DFRL assumes that synthesis failures can be meaningfully partitioned into a small set of *normalized failure categories* (error types), and that configurations sharing the same error type share exploitable structural commonalities. If this assumption is violated (for example, if distinct failure mechanisms produce the same error type, or if a single mechanism produces inconsistent error types across runs), then RPE's per-type signature extraction will degrade. The validity of this assumption is empirical and tool-dependent.

Construction. We define a deterministic keyword-based mapping from raw tool log output to canonical error types. For Bambu: `pipeline_phi_conflict`, `generic_error`, `param_incompatibility`. For Dynamatic: `timeout`, `buffer_placement_failed`. The mapping is tool-specific but deterministic: the same log output always produces the same type.

Known limitations. Under-splitting (different root causes $\rightarrow$ same type) dilutes RPE's ability to find consistent dimensions. Over-splitting (one root cause $\rightarrow$ multiple types) fragments the failure evidence. For example, on Dynamatic the `gemm` benchmark produces both `timeout` and `buffer_placement_failed` errors, limiting RPE's ability to form a single discriminative signature.

Stability verification. We verify cross-run consistency on a subset of 20 configurations per benchmark (same config $\rightarrow$ same error type across repeated runs) and manually spot-check 10 logs per type. The primary safeguard is the mapping's deterministic construction: identical log output always yields the same type, so instability can only arise from non-deterministic tool behavior, which we did not observe in our spot checks. A more exhaustive verification campaign covering the full configuration space would strengthen this assumption but is beyond the scope of the current work.

D. Layer 1: Static Constraint Filtering (SCF)

SCF partitions $\mathcal{C} = \mathcal{C}_{\text{blk}} \cup \mathcal{C}_{\text{sup}} \cup \mathcal{C}_{\text{act}}$ using a small set of hand-crafted rules, each labeled with a *severity* that determines the resulting action.

Sound blocking ($\mathcal{C}_{\text{blk}}$): Permanently removed based on tool documentation (Bambu R1–R2). Provably correct; zero false-positive rate. Within the evidence hierarchy, this is the strongest action, reserved for proven constraints.

Heuristic suppression ($\mathcal{C}_{\text{sup}}$): Provides an *initial risk label*: suppressed configurations are tagged as higher-prior-risk in the initial queue. In the abstract policy this corresponds to active-before-suppressed ordering; in the reported permutation-based experiments (§IV-D), the initial queue is permuted, so suppression acts as a risk label that OFRS may then act on rather than as strict tail placement. Either way, suppression is not a hard exclusion: OFRS may promote a suppressed configuration ahead of others if it earns a lower risk score, and if budget permits all suppressed configurations are eventually reachable.

E. Layer 2: Dynamic Failure Risk Learning (DFRL)

DFRL operates online during exploration, updating its internal state after each synthesis evaluation. It is not a classifier, surrogate model, or machine-learning method: no offline training, no gradient updates, no model fitting. Its state is fully deterministic given $\mathcal{H}_t$ and inspectable at any point.

DFRL requires the failure taxonomy assumption (Section III-C) to hold for RPE, and sufficient observation volume for OFRS.

1) Recurrent Pattern Extractor (RPE): **Semantic scope.** RPE outputs typed recurrent failure signatures: parameter conditions that consistently co-occur with a specific error type and discriminate failures from successes in the current observation history. They are discriminative summaries, not root-cause identifications, and their validity depends on observation coverage and taxonomy stability.

In the evidence hierarchy of Section III-B, RPE operates at the typed-recurrent-evidence level, requiring concentrated, type-specific failure observations before triggering hard skip (the second-strongest action after sound blocking). It relies on structured, repeated evidence rather than marginal or mixed-type statistics.

Signature definition. A signature is a tuple $\pi = (t, \mathbf{cond})$ where t is a normalized error type and $\mathbf{cond} = \{(d_1, v_1), \ldots, (d_k, v_k)\}$ is a set of dimension-value pairs. A configuration c *matches* π, written $c \models \pi$, if and only if $c[d_i] = v_i$ for all $(d_i, v_i) \in \mathbf{cond}$.

The signature's evidence counts are defined over the *entire* current observation history $\mathcal{H}_t$:

$$n_{\text{fail}}(\pi) = |\{(c, y) \in \mathcal{H}_t : y = (\text{fail}, t) \text{ and } c \models \pi\}| \quad (1)$$

$$n_{\text{counter}}(\pi) = |\{(c, y) \in \mathcal{H}_t : y = \text{success and } c \models \pi\}| \quad (2)$$

That is, n_{fail} counts all observed type-t failures matching π, and n_{counter} counts all observed successes matching π, regardless of when they were observed.

Activation boundary. RPE attempts signature extraction only when: (1) $\geq \tau$ failures of the same error type t exist in $\mathcal{H}_t$, and (2) ≥ 1 successful evaluation exists in $\mathcal{H}_t$ (providing contrast for discriminative filtering).

Signature formation. RPE attempts to identify, for each error type, a compact, non-redundant set of parameter conditions that captures the recurrent structure of same-type failures while excluding coincidentally shared but non-informative dimensions. The formation procedure operates as follows:

Discriminative dimension filtering. Given the set of all type-t failures in $\mathcal{H}_t$:

1) **Consistency:** For each dimension d, check whether all type-t failures share the same value v. Retain only dimensions where this holds.
2) **Discrimination:** Among retained dimensions, exclude any where $d = v$ also holds for *every* observation in $\mathcal{H}_t$ (both failures and successes). Such dimensions carry no discriminative information.

3) **Scoring:** The surviving dimensions form a candidate signature $\pi = (t, \mathbf{cond})$ with an evidence score:

$$\text{conf}(\pi) = \frac{n_{\text{fail}}(\pi)}{n_{\text{fail}}(\pi) + n_{\text{counter}}(\pi) + 2} \tag{3}$$

The additive $+2$ in the denominator (without a matching $+1$ in the numerator) implements a conservative prior: with no observations, conf(π)=0, so genuine failure evidence is required to raise the score. This is stronger than the symmetric add-one (Laplace) smoothing used by OFRS in Eq. 4 below, reflecting that RPE's action class (hard skip) is stronger than OFRS's (reorder). Equation 3 is a smoothed empirical evidence score rather than a calibrated probability or formal posterior; it summarizes the concentration of same-type failure evidence relative to counterevidence, and serves as a conservative activation gate that is monotonically increasing in n_{fail} and monotonically decreasing in n_{counter}.

Activation threshold. A signature is activated (triggers hard skip for matching configs) only when conf(π) $\geq \theta$, with $\theta = 0.8$ in our experiments. To give concrete intuition: with $\theta = 0.8$, a signature with zero counterexamples requires $n_{\text{fail}} \geq 8$ to activate ($8/(8+0+2) = 0.8$). With one counterexample, it requires $n_{\text{fail}} \geq 12$ ($12/(12+1+2) = 0.8$). This means hard skip is triggered only after substantial and consistent failure evidence. The threshold θ is a user-configurable parameter; we report a sensitivity sweep across $\theta \in \{0.7, 0.8, 0.9\}$ in Section V-H.

Scope and intended lifecycle. Signatures are *run-scoped*: they apply only within the current exploration run, not across runs or benchmarks. Within a run, the intended lifecycle is *threshold-triggered activation with persistence*: once conf(π) $\geq \theta$, the signature becomes active and is applied to all remaining queue members; the signature remains active as long as conf(π) $\geq \theta$. Since evidence counts are computed over the entire history $\mathcal{H}_t$ (Eqs. 1–2), a signature's confidence can in principle decrease if subsequent observations add to n_{counter}, either through configurations evaluated before the signature was formed, or through probe auditing (below); if conf(π) were to drop below θ, the signature would be deactivated and no longer trigger hard skip. The released implementation logs counter-evidence as an audit trail but does not yet perform live conf(π) recomputation and runtime deactivation; this is deferred to future work (see **Probe auditing** below for why this gap is empirically inert in all reported runs).

The intended lifecycle follows three rules: (1) *activation* is threshold-triggered: a signature becomes active when conf(π) $\geq \theta$; (2) *persistence*: an active signature remains in effect as long as conf(π) $\geq \theta$; (3) *deactivation* occurs only through observed counterevidence that increases n_{counter} and drives conf(π) below θ. Hard-skipped configurations do not directly generate counterevidence because they are not evaluated. The only two sources of counterevidence are: (a) configurations evaluated before the signature was formed that happen to match it and succeeded, and (b) probe evaluations of would-be-skipped configurations.

Probe auditing. To provide a self-correction path, a configuration that would be hard-skipped is instead evaluated with probability p_{probe} (default 0.05). If the probe succeeds, n_{counter} increases and—under the intended lifecycle—conf(π) decreases, potentially deactivating the signature. Note that the probe Bernoulli is re-evaluated each time the skip phase encounters a matching configuration (Algorithm 1, lines 5–11), so a configuration retained as a probe in one iteration may be removed in a subsequent one; in practice only a small expected number of probes actually consume budget. At B=60 (Bambu) and B=30 (Dynamatic), RPE does not activate in the full PA-DSE configuration and no probes are triggered. At B=120 (Bambu), RPE activates and an average of 1.0 probe per run is evaluated, accounting for approximately 1.5% of the budget actually consumed; probe outcomes are logged and counted identically to regular evaluations in all reported metrics. Because no probe succeeds in any reported run (n_{counter} stays at its activation value), the deactivation branch of the lifecycle is never exercised in our experiments, and the gap between the released implementation (which logs counter-evidence but does not recompute conf(π) at runtime) and the intended lifecycle has no effect on any reported number. Implementing live confidence-driven deactivation is a small extension we leave as future work for budgets where probe successes are expected.

Subsumption. Signature $\pi_1 = (t, \mathbf{cond}_1)$ *subsumes* $\pi_2 = (t, \mathbf{cond}_2)$ if: (a) they share the same error type t; (b) $\mathbf{cond}_1 \subsetneq \mathbf{cond}_2$ (π_1 has strictly fewer conditions, all of which appear in $\mathbf{cond}_2$); and (c) π_1 is itself *activated* (conf(π_1) $\geq \theta$). When all three conditions hold, π_2 is deactivated and removed from $\mathcal{P}_t$, because π_1 matches a strict superset of the configurations matched by π_2 and has sufficient evidence to justify hard skip on its own. Condition (c) is critical for conservativeness: a newly formed but insufficiently supported general candidate must not displace an already-activated narrower signature.

2) Online Failure Risk Scorer (OFRS): OFRS is a simple first-order, small-sample risk ranking model whose sole function is queue reordering: it delays configurations with higher estimated failure association but never permanently excludes any configuration. It is not a calibrated failure predictor or classifier.

In the evidence hierarchy of Section III-B, OFRS operates at the weakest level: per-dimension marginal statistics that mix across failure types. This positioning restricts OFRS to reordering rather than permanent exclusion.

Why OFRS only reorders. Unlike RPE, which operates on concentrated, high-evidence per-type signatures, OFRS aggregates marginal single-dimension statistics across all failure types. These marginal statistics are inherently noisy at small sample sizes, cannot capture parameter interactions that may drive failure, and mix evidence from structurally different failure modes. Granting OFRS skip authority would allow noisy marginal evidence to permanently exclude configurations, including potentially Pareto-optimal feasible ones. Restricting OFRS to reordering ensures that all configurations remain reachable if budget permits, and that the only permanent-exclusion mechanism (RPE) requires type-specific, concentrated, high-evidence signatures.

Why first-order is sufficient. HLS infeasibility can involve parameter interactions, and a higher-order model could in

principle capture these. At budgets $B \leq 80$, however, most pairwise $(d_1{=}v_1, d_2{=}v_2)$ combinations have 0–2 observations, making pairwise statistics unreliable. RPE handles interaction effects implicitly through its consistency-based extraction (a signature like {pipeline=True} effectively captures all interactions involving pipeline). OFRS provides marginal ranking improvement as a complement to RPE rather than modelling failure structure independently. In interaction-dominated scenarios where no single dimension separates failure from success, all $w_d \approx 0$ and OFRS produces near-uniform rankings, gracefully degrading to no-op.

Formulation. For each dimension d and value v, the smoothed failure association score is:

$$\hat{f}(d, v) = \frac{n_f(d{=}v) + 1}{n_f(d{=}v) + n_s(d{=}v) + 2} \tag{4}$$

where $n_f(d{=}v)$ and $n_s(d{=}v)$ are the number of observed failures and successes, respectively, with $c[d] = v$. We use $\hat{f}$ rather than $\hat{P}$ to emphasize that this is a smoothed empirical score, not a calibrated probability estimate.

The *relevance* of dimension d is the range of $\hat{f}$ across its observed values:

$$w_d = \max_v \hat{f}(d, v) - \min_v \hat{f}(d, v) \tag{5}$$

Equation 5 includes a cold-start protection: $w_d = 0$ if total observations for dimension d are below $n_{\min} = 5$.

The risk score for configuration c is the relevance-weighted average:

$$r(c) = \begin{cases} \dfrac{\sum_d w_d \cdot \hat{f}(d, c[d])}{\sum_d w_d} & \text{if } \sum_d w_d > 0 \\ 0.5 & \text{otherwise (cold-start fallback)} \end{cases} \tag{6}$$

In Equation 6, when $\sum_d w_d = 0$, OFRS does not alter the queue order; the queue stays in its initial order (which in our experiments is randomized per permutation seed; see §IV-D). As observations accumulate and individual dimensions cross the $n_{\min}$ threshold, OFRS gradually transitions from this passive mode to active global reordering. This transition is smooth: each dimension independently contributes to ranking only after it has accumulated sufficient evidence, so OFRS's influence grows incrementally rather than switching on abruptly.

F. Module Interface Specification

The two layers yield four distinct operations that interact through a well-defined priority ordering, each reflecting the evidence hierarchy principle that action strength must not exceed evidence strength:

1. Sound blocking (SCF) permanently removes configurations from the design space before exploration begins. This action is irreversible and provably correct (zero false positives). *Evidence: proven constraints. Action: strongest (permanent block).*

2. Heuristic suppression (SCF) establishes an *initial risk label*: suppressed configurations carry a higher-prior-risk tag that OFRS may act on. This is *not* a hard exclusion; it is a label that OFRS may override as evidence accumulates. In practice, suppressed configurations typically receive high risk scores from OFRS (because they match failure-prone parameter combinations), so OFRS tends to reinforce rather than contradict the static prior. *Evidence: medium (empirical). Action: risk label, overridable.*

3. RPE hard skip removes matching configurations from the queue during exploration. RPE operates independently of OFRS: it removes configs before OFRS ranks the remainder. A configuration may be simultaneously suppressed (by SCF) and RPE-skipped; the skip takes precedence. *Evidence: high (empirical, typed, concentrated). Action: run-scoped permanent removal.*

4. OFRS reordering sorts the remaining queue by risk score. This is a global sort over all remaining configurations, both active and suppressed. OFRS reordering is the weakest action: it never removes configurations, only changes their evaluation order. *Evidence: low–medium (marginal, mixed-type). Action: weakest (reorder only).*

Conflict handling. OFRS may promote a heuristically-suppressed configuration ahead of active ones if its risk score is lower. OFRS reacts to actual synthesis evidence, which may reveal that a suppressed configuration's non-suppression parameters are associated with success. Since OFRS only promotes (evaluates earlier, never skips), the worst case is one wasted budget unit, which is preferable to permanently excluding a potentially feasible design.

Cold-start window. During the first $\sim n_{\min}$ evaluations, OFRS has insufficient data for meaningful relevance scores. During this period, queue order is the initial order (permuted in the reported experiments; see §IV-D). RPE also requires $\geq \tau$ same-type failures before activating, so early exploration proceeds without dynamic intervention.

G. Exploration Algorithm

Algorithm 1 summarizes the PA-DSE policy; Figure 1 shows the overall two-layer structure and the evidence-hierarchy bounds on each component's action class. Newly learned RPE signatures take effect in the next iteration of the while loop, not retroactively.

How $n_{\text{counter}}(\pi)$ is maintained. Algorithm 1 shows an explicit `RPE.addCounter` call only on probe-success paths (inside the `if probe` sub-branch after a successful synthesis). This is consistent with Eq. 2 because the two sources of counterevidence identified in §III-E1 are handled at different points. Pre-signature counterevidence (case (a): successes in $\mathcal{H}_t$ that happen to match π) is computed by `RPE.tryExtract` when the signature is first evaluated for activation: the routine scans the success log $\mathcal{H}_t^{\text{succ}}$ and counts matches against the candidate π. Post-activation counterevidence (case (b): probe successes) is added incrementally by the `addCounter` call. Non-probe successes after activation cannot match an active signature by construction: every config matching an active signature is routed through the probe-or-skip branch in the RPE filter (the `if RPE.matches` block before synthesis), so any non-probe success at synthesis time is by definition a config that did not match any active signature, and there is nothing to update.

Algorithm 1 PA-DSE Exploration Policy

Require: $\mathcal{C}$, source S, budget B, threshold τ, probe rate p_{probe}
Ensure: Results $\mathcal{R}$

$\mathcal{C}_{\text{act}}, \mathcal{C}_{\text{blk}}, \mathcal{C}_{\text{sup}} \leftarrow \text{SCF}(\mathcal{C}, S)$
$Q \leftarrow \mathcal{C}_{\text{act}} \,\|\, \mathcal{C}_{\text{sup}}$; $n \leftarrow 0$ {In experiments Q is then queue-permuted (§IV-D); suppression remains a risk label rather than strict tail placement.}
while $Q \neq \emptyset$ **and** $n < B$ **do**
  for $c \in Q$ **do**
    if RPE.matches(c) **then**
      if rand() $< p_{\text{probe}}$ **then**
        mark c as probe
      else
        remove c from Q {Hard skip}
      end if
    end if
  end for
  Sort Q by $r(c)$ ascending {OFRS: global reorder}
  $c \leftarrow Q$.popFirst(); $n \leftarrow n + 1$
  $(\text{ok}, \text{met}, \text{out}) \leftarrow \text{Synthesize}(c, S)$
  if ok **then**
    OFRS.update(c, success)
    if c is probe **then**
      RPE.addCounter(c)
    end if
  else
    OFRS.update(c, failure)
    RPE.tryExtract(out, c, τ)
  end if
  $\mathcal{R} \leftarrow \mathcal{R} \cup \{(c, \text{met}, \text{ok})\}$
end while
return $\mathcal{R}$

IV. Experimental Setup

A. Tools and Benchmarks

We evaluate PA-DSE on two HLS tools that differ fundamentally in scheduling philosophy:

- **PandA-Bambu v0.9.8** [3]: statically scheduled, C-centric, with a well-established failure-taxonomy surface.
- **Dynamatic v2.0** [4]: dynamically scheduled (elastic dataflow), LLVM-based, with MILP-driven buffer placement (solved using Gurobi 12.0 in our setup).

This pairing stresses tool-generality: the two tools share no pragma vocabulary, expose disjoint failure modes, and benefit asymmetrically from static versus dynamic pruning, as we analyze below.

Benchmarks. Both tools are evaluated on the same eight benchmarks: `matmul`, `vadd`, `fir`, `histogram`, `atax`, `bicg`, `gemm`, and `gesummv`. The selection combines Polybench linear-algebra kernels and simple DSP kernels, covering a range of loop-nest structures and feasibility landscapes. We chose benchmarks that are present in both tools' integration-test suites to enable cross-tool comparison; CHStone and MachSuite benchmarks were considered but require non-trivial porting to Dynamatic's LLVM-based frontend. Using the same benchmark set on both tools enables direct cross-tool comparison of failure-mode structure and PA-DSE behavior. (Dynamatic's integration-test suite names the matrix-multiplication kernel `matrix`; we use `matmul` throughout for consistency.)

B. Configuration Spaces

Bambu (420 configurations). The enumerated space is the Cartesian product of clock period (`clock_period` $\in \{3, 5, 8, 10, 15, 20, 25\}$, in ns), memory allocation policy (`memory_policy` $\in$ {`ALL_BRAM`, `NO_BRAM`}), memory controller type (`channels_type` $\in$ {`MEM_ACC_11`, `MEM_ACC_N1`, `MEM_ACC_NN`}), channel count (`channels_number` $\in \{1, 2\}$), and a loop-pipelining axis that takes one of five values: *off* (no pipelining pragma issued) or *on* with initiation interval II $\in \{1, 2, 4, 8\}$. The full enumeration thus has $7 \times 2 \times 3 \times 2 \times (1 + 4) = 420$ configurations: $7 \times 2 \times 3 \times 2 = 84$ with pipelining off, and $7 \times 4 \times 2 \times 3 \times 2 = 336$ with pipelining on, which is the 336/420 $\approx 80\%$ pipeline-enabled fraction referenced in §V-I. Two tool-documented incompatibilities contribute a provably infeasible *blocked* subset $\mathcal{C}_{\text{blk}}$ of size 140 (33.3%): MEM_ACC_11 requires exactly one channel, MEM_ACC_NN requires at least two; these rules combine with the other dimensions to remove 112 pipeline-enabled and 28 pipeline-off configurations, leaving 224 pipeline-enabled configurations in the unblocked queue (used in §V-I). In addition, source-level heuristics mark these same 224 pipeline-enabled configurations as *suppressed* for a subset of benchmarks (`matmul`, `fir`, `histogram`); these are deferred but not removed from the queue.

Dynamatic (192 configurations). The enumerated space covers target clock period, buffer algorithm (`on_merges`, `fpga20`, `fpl22`), hardware sharing, LSQ sizing, and fast-token-delivery tuning. *No tool-documented incompatibility rules apply to this space,* so under SCF all 192 configurations remain in the active set: $\mathcal{C}_{\text{blk}} = \emptyset$ and $\mathcal{C}_{\text{sup}} = \emptyset$. SCF-on and SCF-off behave identically on Dynamatic, making this tool a deliberate stress test of whether DFRL can carry the feasibility-aware load on its own (Section V-G).

Feasibility landscape asymmetry. Three of the eight benchmarks (`vadd`, `fir`, and `histogram`) achieve 100% ground-truth feasibility on Dynamatic: all 192 configurations synthesize successfully regardless of buffer algorithm, clock period, or sharing options. This contrasts with Bambu, where the same three benchmarks exhibit the same $\sim$15% random-sampling SR as all other benchmarks, because Bambu's dominant infeasibility sources—MEM_ACC parameter incompatibilities and the pipeline-enabled high-risk region—are pragma-level and source-independent.

The difference reflects a structural asymmetry in failure mechanisms. On Bambu, infeasibility is *pragma-driven*: tool-level parameter rules (R1–R2) reject configurations regardless of source complexity. On Dynamatic, infeasibility is *graph-complexity-driven*: the MILP-based buffer-placement solver's difficulty scales with the dataflow graph's back-edge count and path interactions, so simple kernels with flat loop structures are trivially solvable while complex kernels (`gemm`, ground-truth SR 35.4%; `matmul`, 45.8%) produce substantial failure

fractions. This asymmetry motivates the two-layer design: SCF addresses documented pragma constraints, while DFRL handles residual pragma-driven (Bambu pipeline region) and graph-complexity-driven (Dynamatic) failures. Because the three trivially feasible benchmarks carry no discriminative signal (all methods achieve 100% SR), we report aggregate Dynamatic metrics over the five non-trivial benchmarks (`matmul`, `atax`, `bicg`, `gemm`, `gesummv`). Per-benchmark results for all eight benchmarks are shown in Figure 5 for completeness.

C. Baselines

We compare against six baselines chosen to span the classical and learned HLS DSE landscape:

- **Random**: uniform sampling without replacement from the full $\mathcal{C}$, providing a no-prior reference that exercises the entire enumerated grid including configurations later shown to be statically infeasible.
- **Filtered Random**: uniform sampling from $\mathcal{C} \setminus \mathcal{C}_{\text{blk}}$, isolating the contribution of static filtering alone. On Dynamatic this reduces to Random since $\mathcal{C}_{\text{blk}} = \emptyset$.
- **Simulated Annealing (SA)** and **Genetic Algorithm (GA)**: classical black-box optimizers using a failure-penalized area/component-count objective (successful syntheses are scored by QoR; failures receive a penalty larger than any feasible score) [6], [7].
- **Bayesian Optimization (GP-BO)**: Gaussian-process surrogate with expected-improvement acquisition, following the BO-for-HLS-DSE line of work [2], [5].
- **Random Forest Classifier (RF)**: an online random-forest surrogate that combines a binary feasibility classifier with a QoR regressor ($n_{\text{estimators}}$=50, max_depth=8). The first 10 configurations are evaluated randomly to collect an initial training set; thereafter the model is retrained every 5 evaluations. At each step, the classifier predicts $\hat{p}(\text{feasible})$ and the regressor predicts normalized QoR for all remaining configurations; these are combined into a score $\hat{p} - 0.3\,\hat{q}_{\text{norm}}$ and configurations are evaluated in descending score order. This is a simplified online proxy inspired by the scalable surrogate of AutoScaleDSE [1]; it differs from the original in that it trains online within a single run rather than on an offline corpus across designs.

Trajectory-based baselines (SA, GA, GP-BO, RF) operate on the same unblocked configuration set $\mathcal{C} \setminus \mathcal{C}_{\text{blk}}$ as Filtered Random, so that any improvement over Filtered Random isolates the contribution of the optimizer's online learning rather than the static filter. Random alone retains the raw $\mathcal{C}$ to anchor the no-prior reference point. All baselines share the same synthesis interface and the same per-run budget. On Dynamatic, $\mathcal{C}_{\text{blk}} = \emptyset$ so every baseline operates on the same $\mathcal{C}$.

D. Evaluation Protocol

Budgets. We use B=60 evaluations per run on Bambu and B=30 on Dynamatic. These are chosen to represent realistic "coffee-break" interactive budgets: with per-call mean synthesis cost of ≈ 2.1 s (Bambu) and ≈ 16.5 s (Dynamatic), a full run completes in roughly 2 min on Bambu and 8 min on Dynamatic. The ablation in Section V-G reports B=30 on both tools as the principal cell; a separate B=120 extended-budget run on Bambu is reported in Section V-I to expose the regime in which RPE activates.

Repetitions. Each (method, benchmark, budget) cell is repeated 10 times. For stochastic baselines (Random, SA, GA, GP-BO, RF) the repetition index is a random seed. For deterministic methods (Filtered Random, PA-DSE) we vary the initial queue ordering via a `queue_permutation_id`, which is mechanically equivalent to a seed for sensitivity analysis. This isolates ordering sensitivity from optimizer-internal randomness.

PA-DSE hyperparameters. We freeze τ=2 (minimum failure support for RPE signature activation), θ=0.8 (RPE confidence threshold), $n_{\min}$=5 (OFRS cold-start observation threshold), and p_{probe}=0.05 (probe rate for keeping active signatures honest). These defaults are not tuned per-benchmark or per-tool.

Metrics. We report:

- **SR (%)**: fraction of evaluated configurations that synthesized successfully, the primary feasibility metric.
- **Wasted calls**: count of synthesis invocations that returned a failure, a proxy for user-visible time cost.
- **TTFF (s)**: wall-clock time to first feasible design, relevant for interactive use.
- **UQoR**: number of unique $(\text{area}, \text{latency})$ pairs discovered across successful evaluations in a run (Python-set deduplication on the raw QoR tuple). Tables III–IV report the mean across runs. This is a measure of QoR-space coverage breadth; Pareto-front coverage is reported separately in Fig. 7 (§V-F).
- **Best area, best latency**: minima observed across successful configurations in the run.
- **Algorithmic overhead**: per-iteration time spent in each PA-DSE layer (`overhead_scf_ms`, `overhead_rpe_ms`, `overhead_ofrs_ms`).

Hardware. All runs execute single-threaded on an Azure `Standard_D4s_v5` VM (4 vCPUs, 15 GB RAM, Ubuntu 24.04, Python 3.12). No GPUs are used. Gurobi 12.0 (academic license) is used for Dynamatic's MILP buffer placement.

Reproducibility. Code, configuration generators, benchmark sources, and aggregated run summaries are released at https://github.com/jane09250410/hls-dse. All tables and most figures can be regenerated from the included `run_summary.csv` files; two figures requiring per-evaluation traces (convergence and QoR plots) require experiment reruns to produce the underlying `eval_log.csv` files.

V. Results

The evaluation addresses nine questions in turn: whether PA-DSE outperforms classical and learned baselines on success rate (§V-A, §V-B); how large the user-visible cost savings are in wasted calls and TTFF (§V-C); whether per-benchmark results are consistent or driven by a few easy cases (§V-D);

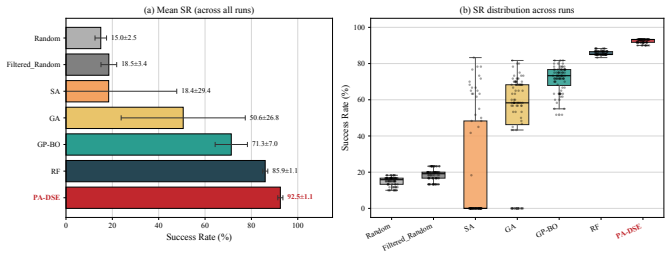

Fig. 2. Bambu main results (B=60). (a) Mean SR with standard deviation error bars. (b) Per-run SR distribution: PA-DSE matches the tightest σ (1.1) among methods with non-trivial SR, while reaching the highest mean.

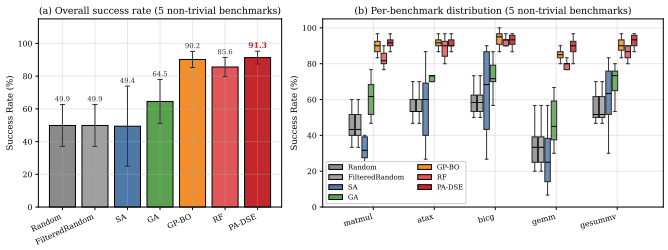

Fig. 3. Dynamatic main results (B=30). (a) Aggregate SR with standard deviation across five non-trivial benchmarks. (b) Per-benchmark SR distribution. PA-DSE achieves the highest aggregate SR and remains competitive across non-trivial Dynamatic benchmarks, with lower variance among deployable methods.

what the convergence trajectory looks like (§V-E); whether queue reordering sacrifices QoR quality (§V-F); whether the ablation confirms the evidence-hierarchy design (§V-G); how sensitive PA-DSE is to the RPE confidence threshold θ (§V-H); whether RPE contributes at extended budgets where it activates (§V-I); and whether the algorithmic overhead is negligible relative to synthesis cost (§V-J).

A. Main Comparison on Bambu

Table III reports per-method means and standard deviations across all $(benchmark, \text{run})$ cells on Bambu at B=60. PA-DSE achieves 92.5%±1.1 success rate, exceeding the strongest baseline (RF, $85.9\% \pm 1.1$) by 6.6 pp. More importantly, PA-DSE wastes only 4.5 synthesis calls on average per run, compared to 8.4 for RF, 17.2 for GP-BO, and 51.0 for Random, a $\mathbf{1.9}\times$ reduction over the best baseline and an $\mathbf{11}\times$ reduction over Random. Because each wasted call costs ≈ 2.1 s of user time on Bambu, PA-DSE saves roughly 8 s of wall clock per run versus RF.

Two further points are worth noting. PA-DSE shows minimal sensitivity to initial queue permutation, with σ=1.1 across 10 permutations on Bambu. RF reports σ=1.1 across 10 seeds; the two σ values measure different sources of variation (permutation-induced ordering vs. optimizer-internal randomness) and should not be compared directly as a robustness claim. Simulated annealing, by contrast, has σ=29.4: some runs find feasible configurations early and succeed, while others get trapped in infeasible regions, consistent with the broader pattern that classical black-box methods are ill-suited to design spaces dominated by structural infeasibility. On Bambu PA-DSE also matches the best area of every baseline (464.1, tied at the tool's lower bound), achieves the lowest best latency among all methods (10.25 cycles), and the highest UQoR (19.1); the cross-tool picture is more nuanced and is discussed in §V-B. Figure 2 visualizes the full per-method distribution.

B. Cross-Tool Evaluation on Dynamatic

Table IV reports results on Dynamatic at B=30, aggregated over the five benchmarks with non-trivial failure rates. Because Dynamatic exposes no tool-documented incompatibility rules ($\mathcal{C}_{\text{blk}} = \emptyset$), SCF reduces to a no-op; Filtered Random is identical to Random, and the SCF-only ablation matches no-filter (Section V-G). Any improvement over Random must therefore come from DFRL.

PA-DSE achieves 91.3% SR, compared to GP-BO (90.2%) and RF (85.6%). The more distinctive advantage over GP-BO is in user-visible cost: TTFF drops to 34.2 s versus 67.0 s for GP-BO and RF, roughly half the wall-clock time, and Wasted drops to 2.6 versus 2.9 and 4.3. For interactive developers iterating tens of times a day, this compounds to significant time savings. Figure 3 shows the per-benchmark distribution.

Reading the QoR columns. The last three columns (UQoR, best area, best latency) require careful reading. Among methods with SR $\geq 85\%$ (GP-BO, RF, and PA-DSE), best latency values are comparable: PA-DSE 407.6, GP-BO 405.4, RF 413.0. The lat-stronger baselines (Random 404.0, GA 403.7) sample the design space broadly; this broad sampling occasionally discovers low-latency configurations that PA-DSE does not reach within the limited budget, but the same broad sampling is what produces their catastrophic SR (49.9%, 64.5%) and Wasted counts (10–15). Restated: *a method that wastes one third to one half of its budget on infeasible configurations does occasionally bump into a low-latency point, but is not a viable production policy*. PA-DSE's UQoR (7.1) leads all SR $\geq 85\%$ baselines (GP-BO 5.1, RF 6.1), indicating that feasibility-focused queue reordering does not sacrifice QoR diversity.

C. User-Visible Cost: Wasted Calls and Time-to-First-Feasible

Success rate is only one dimension of user experience; two others dominate perceived quality in interactive DSE: how many synthesis calls are wasted (each wasted call is a direct time cost) and how quickly the first feasible design is found (influences iteration speed). Figure 4 summarizes both.

PA-DSE wastes 4.5 synthesis calls per run on Bambu (11.3× fewer than Random, 1.9× fewer than RF) and 2.6 on Dynamatic (vs. 2.9 for GP-BO and 4.3 for RF). Translated to wall-clock: on Bambu, PA-DSE saves about 8 s per run versus RF (≈ 4 calls $\times$ 2.1 s); on Dynamatic, about 5 s per run versus GP-BO (≈ 0.3 calls $\times$ 16.5 s) and about 28 s versus RF (≈ 1.7 calls $\times$ 16.5 s). For interactive developers iterating tens of times a day, this compounds to meaningful time savings. TTFF shows a similar picture on Dynamatic (34.2 s for PA-DSE vs. 67.0 s for GP-BO/RF); on Bambu, classical methods have unusually low TTFF because they find occasional trivial successes early, but their mean SR is much lower (Table III).

TABLE III
MAIN RESULTS ON BAMBU (B=60, n=80 RUNS PER METHOD: 8 BENCHMARKS × 10 SEEDS/PERMUTATIONS). PA-DSE'S σ REFLECTS QUEUE-PERMUTATION VARIATION; OTHER METHODS' σ REFLECTS SEED VARIATION.

Method	SR (%)	Wasted	TTFF (s)	UQoR	Best area	Best latency
Random	15.0 ±2.5	51.0	22.9	6.4	473.1	10.94
Filtered Random	18.5 ±3.4	48.9	11.9	7.9	473.9	10.94
SA	18.4 ±29.4	49.0	2.5	4.3	464.1	10.58
GA	50.6 ±26.8	29.6	7.7	11.8	464.1	10.61
GP-BO	71.3 ±7.0	17.2	10.9	14.9	464.1	10.28
RF	85.9 ±1.1	8.4	14.3	17.1	464.1	10.31
PA-DSE	**92.5** ±1.1	**4.5**	8.9	**19.1**	**464.1**	**10.25**

TABLE IV
MAIN RESULTS ON DYNAMATIC (B=30, n=50 RUNS PER METHOD: 5 NON-TRIVIAL BENCHMARKS × 10 SEEDS/PERMUTATIONS). FILTERED RANDOM = RANDOM BECAUSE $\mathcal{C}_{\text{BLK}} = \emptyset$. THREE BENCHMARKS (`vadd`, `fir`, `histogram`) ACHIEVE 100% SR FOR ALL METHODS AND ARE EXCLUDED FROM AGGREGATION.

Method	SR (%)	Wasted	TTFF (s)	UQoR	Best area	Best latency
Random	49.9 ±12.8	15.0	36.6	9.3	51.7	404.0
Filtered Random	49.9 ±12.8	15.0	36.6	9.3	51.7	404.0
SA	49.4 ±24.5	15.2	64.7	7.4	52.8	408.7
GA	64.5 ±13.4	10.6	36.6	8.5	51.2	403.7
GP-BO	90.2 ±4.9	2.9	67.0	5.1	51.3	405.4
RF	85.6 ±5.8	4.3	67.0	6.1	51.3	413.0
PA-DSE	**91.3** ±3.9	**2.6**	**34.2**	7.1	51.3	407.6

D. Per-Benchmark Consistency

Aggregate SR can mask method-benchmark interactions: a method might win on average while failing badly on a specific benchmark. Figure 5 plots mean SR for every (method, benchmark) cell.

PA-DSE achieves $\geq 90\%$ SR on every Bambu benchmark, while every baseline (including RF) has at least one benchmark where it drops noticeably. SA and GA in particular show high per-benchmark variance, consistent with their sensitivity to the benchmark-specific feasibility landscape. On Dynamatic, per-benchmark variation is more pronounced than on Bambu, reflecting the graph-complexity-driven nature of Dynamatic's failure modes. Three benchmarks (`vadd`, `fir`, `histogram`) achieve 100% SR for all methods due to their trivially simple dataflow graphs. Among the five non-trivial benchmarks, `gemm` is the hardest (ground-truth feasibility 35.4%), and PA-DSE achieves 89.3% SR, outperforming GP-BO and RF by the widest margins in the evaluation. This pattern (PA-DSE's advantage growing on harder benchmarks) is consistent with OFRS learning the dominant failure dimension (`buffer_algorithm`: `on-merges` achieves 0% failure rate, while `fpl22` reaches 82.5%) early and concentrating budget on feasible configurations.

a) Why some Bambu rows are nearly benchmark-agnostic.: Several rows in Figure 5(a) (Random, Filtered Random, RF, and PA-DSE) show essentially identical SR across all eight benchmarks. This is structural, not an artifact, and results from three properties acting together: (1) the dominant infeasibility band ($\mathcal{C}_{\text{blk}}$, 33% of $\mathcal{C}$) is determined by tool-level MEM_ACC rules that are independent of source code; (2) these methods derive their evaluation order from configuration-level signals only—a fixed seed/permutation in the case of Random and Filtered Random, and online configuration-level success/failure feedback in the case of RF and PA-DSE—without consulting benchmark-specific source features, and because the Bambu ground-truth feasibility label is itself benchmark-agnostic (every benchmark has the same $56/420$ feasible configurations), the success/failure traces these methods observe are nearly identical across benchmarks; and (3) at B=60, the budget is small relative to the unblocked space $\mathcal{C} \setminus \mathcal{C}_{\text{blk}}$ (size 280). For Filtered Random, RF, and PA-DSE—which sample from $\mathcal{C} \setminus \mathcal{C}_{\text{blk}}$—the evaluated calls land in this unblocked space, whose composition is shared across benchmarks. For Random, the calls land uniformly in the full $\mathcal{C}$ with the expected ∼33% rate falling in $\mathcal{C}_{\text{blk}}$, but $\mathcal{C}_{\text{blk}}$ itself is benchmark-independent (defined by tool-level MEM_ACC rules), so even Random's failure pattern is uniform across benchmarks. Within the unblocked space, the smaller suppressed subset $\mathcal{C}_{\text{sup}}$ (reached on `matmul`/`fir`/`histogram`) is a risk label rather than a hard exclusion (Algorithm 1, line

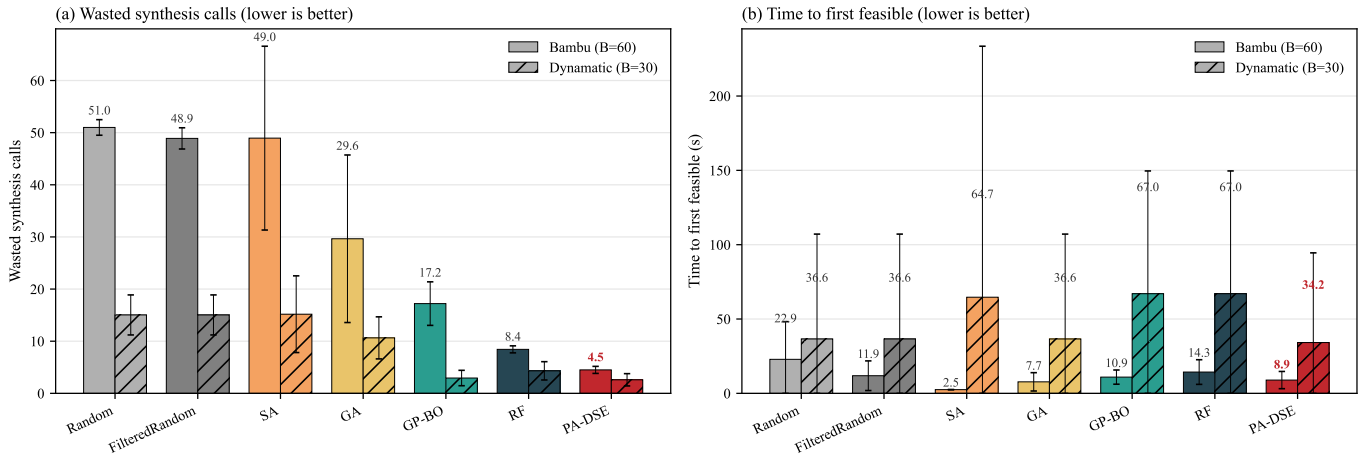

Fig. 4. User-visible cost. (a) Wasted synthesis calls per run (lower is better): PA-DSE wastes 4.5 on Bambu (1.9× fewer than RF) and 2.6 on Dynamatic (vs. 2.9 for GP-BO and 4.3 for RF). (b) Time to first feasible (lower is better): on Dynamatic, PA-DSE reaches the first feasible design in half the wall-clock time of GP-BO and RF.

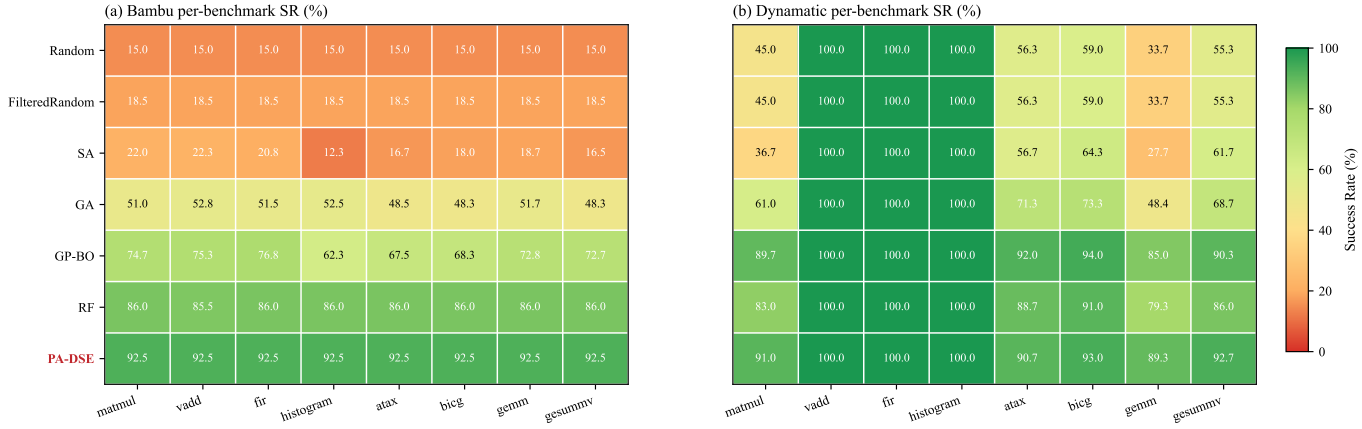

Fig. 5. Per-benchmark success rate (%). PA-DSE is consistently high across benchmarks: it wins or ties on all Bambu benchmarks and remains competitive on all Dynamatic benchmarks, with low per-run variance. Several rows in panel (a) appear flat; this is structural, not an artifact (§V-D).

2), so it does not produce systematic per-benchmark variation for these methods. By contrast, GP-BO, SA, and GA carry benchmark-specific internal state that produces genuine per-benchmark variation. On Dynamatic, failure modes are source-dependent, so PA-DSE's per-benchmark SR varies across benchmarks depending on graph complexity.

E. Convergence Behavior

Figure 6 shows the cumulative number of feasible designs found as a function of the evaluation step, averaged across runs. This reveals not just the *amount* of budget used but the *trajectory*: does PA-DSE front-load its successes or find them throughout?

On Bambu (panel a), PA-DSE's curve rises nearly linearly through the full 60-step budget, reflecting that DFRL continually surfaces feasible configurations rather than finding them all early and then stalling. RF catches up toward the tail; SA and GA exhibit characteristic stagnation. On Dynamatic (panel b), PA-DSE converges within the first $\sim$ 20 steps to $\approx$ 28 feasibles, while classical methods ($<$ 20 feasibles at step 30) never catch up.

F. Quality of Results (QoR)

Finding feasible configurations is necessary but not sufficient; users ultimately want coverage of the (area, latency) quality space, especially near the Pareto front. We evaluate QoR coverage by overlaying the unique (area, latency) points discovered by PA-DSE and the union of all baselines. Each tool's plot in Figure 7 categorizes every unique QoR point by which method(s) found it: *shared* (both PA-DSE and at least one baseline), *baseline-only* (no PA-DSE run found it), and Pareto-front markers (large diamonds). The annotation in each panel reports PA-DSE's coverage of the Pareto-optimal vertices.

Three patterns emerge. The *shared* regions dominate both panels, indicating that PA-DSE rediscovers most of what the baselines find collectively. *Baseline-only* points are sparse on both tools and absent on Bambu, so PA-DSE's feasibility focus does not create systematic blind spots in the QoR space. PA-DSE also reaches every Pareto-optimal point on both tools (Pareto coverage 2/2 on Bambu and 4/4 on Dynamatic; see annotations on each panel). Reliability and QoR coverage do

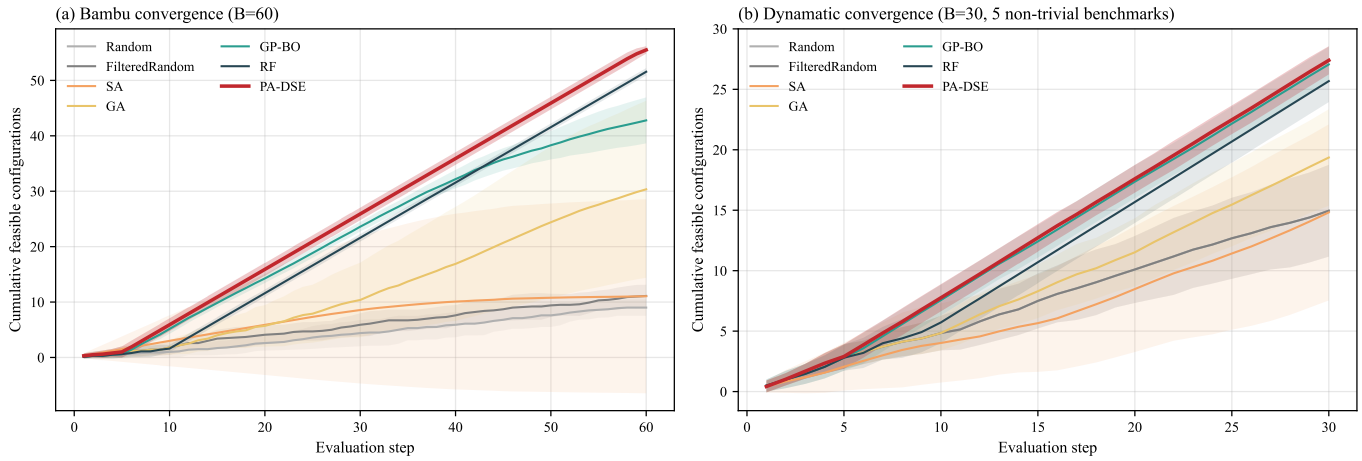

Fig. 6. Convergence curves (mean $\pm 1\sigma$ across runs). (a) Bambu at B=60: PA-DSE maintains near-linear progression throughout the budget, indicating that OFRS's online reordering keeps promising configurations at the queue front. (b) Dynamatic at B=30: PA-DSE's curve is steeper than the classical baselines, reflecting rapid learning of the dominant failure dimension.

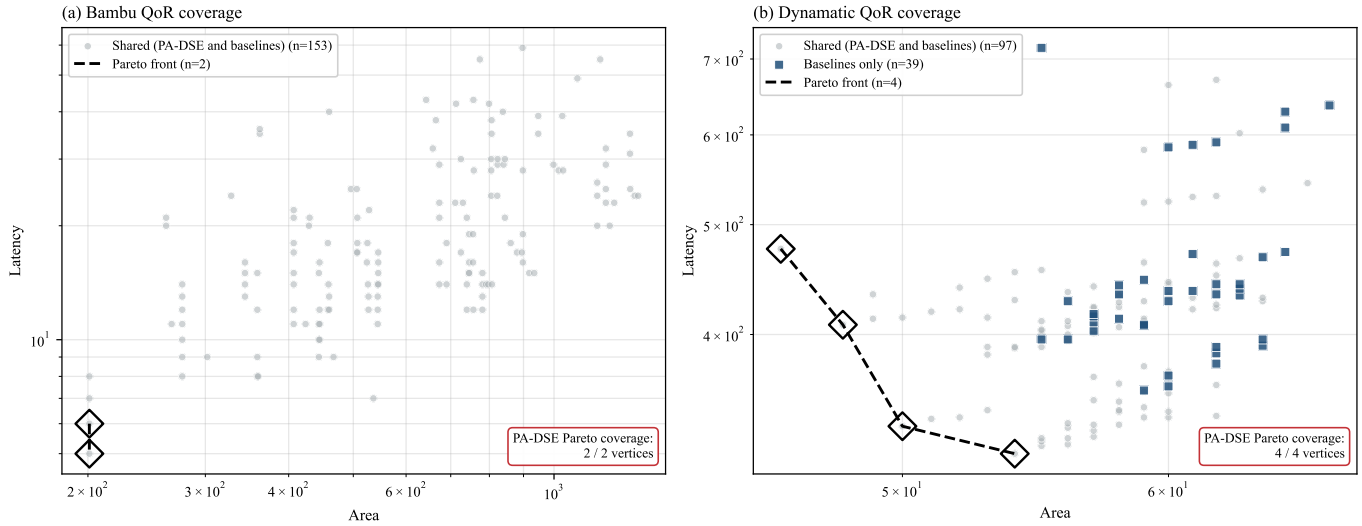

Fig. 7. QoR coverage comparison: each panel overlays the unique (area, latency) points discovered by PA-DSE and the union of baselines. *Shared* points (gray) are found by both; *baseline-only* points (blue) are missed by PA-DSE; Pareto-optimal vertices are marked with large diamonds. PA-DSE covers all Pareto-front vertices on both tools.

not trade against each other in our setting.

G. Ablation: Isolating the Contribution of Each Component

Table V reports an 8-way ablation at B=30 on both tools, separating SCF, RPE, and OFRS and probing the effect of changing OFRS's action class from *reorder* to *skip*. Figure 8 visualizes the same data as a grouped bar chart. Several findings emerge:

a) OFRS is the dominant contributor to SR.: On Bambu, adding OFRS to SCF drives SR from 17.8% (SCF-only) to 84.5% (SCF+OFRS), a +66.7 pp gain. On Dynamatic the effect is similar: +41.5 pp from SCF-only (48.1%) to SCF+OFRS (89.6%). Across both tools, OFRS's online queue reordering is doing the heavy lifting at these budgets.

b) SCF provides a safety net on tools with static rules.: Comparing SCF+DFRL to DFRL-only isolates SCF's marginal value. On Bambu, SCF adds +4.5 pp (80.0% → 84.5%) by removing provably infeasible configurations before DFRL can be contaminated by their failures. On Dynamatic, SCF adds 0 pp because $\mathcal{C}_{\text{blk}} = \emptyset$. SCF's benefit is thus contingent on the tool exposing documented rules, but is non-negative: it never hurts.

At the budgets considered here, RPE does not contribute on top of OFRS: SCF+DFRL and SCF+OFRS are numerically identical (84.5% on Bambu, 89.6% on Dynamatic), yet SCF+RPE alone achieves only 57.8% on Bambu, well below SCF+OFRS. Inspection of per-run logs reveals the mechanism: when OFRS is active, it reorders high-risk pipeline-enabled configurations to the tail of the queue. These are precisely the configurations RPE would have extracted signatures from. At B=30, OFRS's reordering means the budget is exhausted before the queue advances to where RPE would activate. The components are functional; they are not in use at this budget.

TABLE V
ABLATION ON BOTH TOOLS (B=30, BAMBU n=24, DYNAMATIC n=25 PER CONFIG). CONFIGURATIONS ARE GROUPED BY WHICH COMPONENTS ARE ACTIVE.

Configuration	Bambu		Dynamatic	
	SR (%)	Wasted	SR (%)	Wasted
no-filter	12.2 ±9.8	26.3	48.1 ±14.9	15.6
SCF-only	17.8 ±6.5	24.7	48.1 ±14.9	15.6
SCF + RPE	57.8 ±1.6	12.7	48.1 ±14.9	15.6
SCF + OFRS	84.5 ±3.2	4.7	89.6 ±4.1	3.1
SCF + DFRL (full)	**84.5** ±3.2	**4.7**	**89.6** ±4.1	**3.1**
DFRL-only	80.0 ±7.4	6.0	89.6 ±4.1	3.1
SCF + RPE-as-reorder	84.5 ±3.2	4.7	89.6 ±4.1	3.1
SCF + OFRS-as-skip (unsafe)	28.9 ±41.8	2.0	7.7 ±26.8	1.0

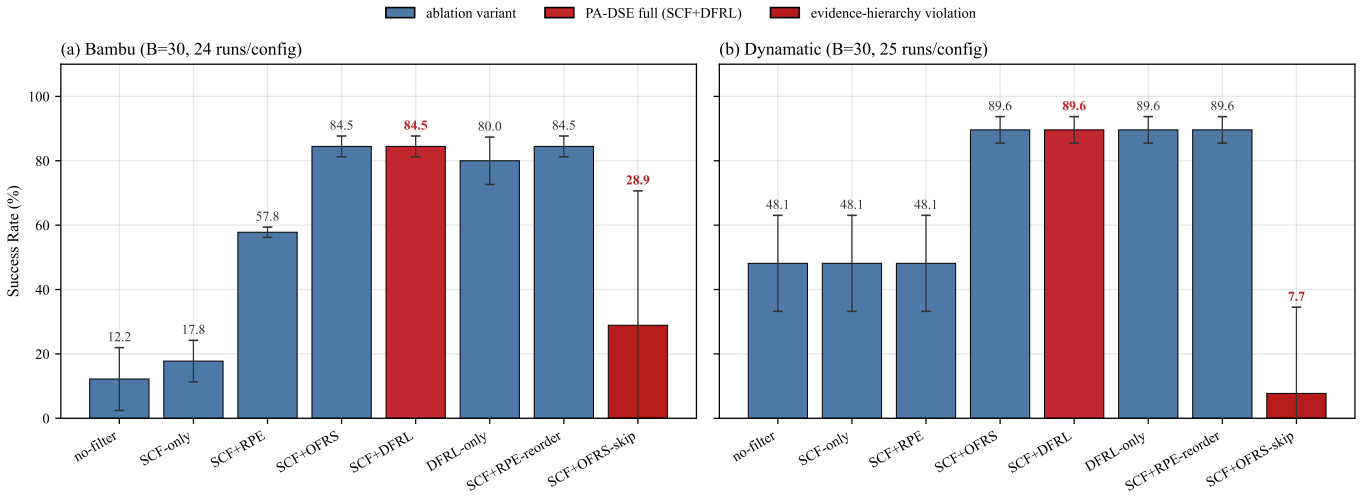

Fig. 8. Ablation study visualization (B=30). PA-DSE full configuration (SCF+DFRL) highlighted in red. The rightmost bar (SCF+OFRS-skip, dark red) violates the evidence hierarchy by granting OFRS skip authority based on weak per-dimension evidence, producing order-of-magnitude larger variance and mean SR collapse on both tools.

Section V-I confirms this: at B=120, RPE activates and drives a +36.3 pp SR gain over SCF+OFRS alone (83.0% vs. 46.7%, $p < 0.001$). Table VI further shows that the RPE confidence threshold θ is a meaningful lever at this budget, with an 18.8 pp SR range across $\theta \in \{0.7, 0.8, 0.9\}$.

A complementary check swaps RPE's action from *skip* to *reorder*, which yields identical results to SCF+OFRS (84.5% on Bambu). This is not a contradiction: when OFRS is already reordering by failure risk, adding RPE's risk-based reorder is redundant. The ablation confirms that RPE's distinct contribution is its *skip* action, not any reordering signal.

c) Allowing OFRS to skip catastrophically increases variance.: The SCF+OFRS-as-skip configuration, which permits OFRS to hard-skip configurations at a risk threshold of 0.85, produces mean SR 28.9% with σ=41.8 on Bambu and 7.7% with σ=26.8 on Dynamatic—variances an order of magnitude larger than any other configuration. On some runs OFRS's first-order marginal statistics happen to align with true failure causes and the run succeeds; on others, OFRS prunes feasible configurations based on noisy single-dimension correlations and the run collapses. Granting marginal statistics skip authority, without the consistency-and-contrast guards that RPE imposes (type-specific filtering, discriminative dimension selection, conservatively-smoothed confidence threshold), produces catastrophic outcomes. The specific safeguards RPE provides are load-bearing.

d) Summary.: The ablation confirms the evidence hierarchy and that no tier is removable. (i) SCF's permanent block is safe because it uses provable evidence; removing the SCF tier from full PA-DSE (DFRL-only row) costs 4.5 pp on Bambu. (ii) OFRS's reordering is safe despite weak evidence because the action itself is weak; removing the OFRS tier (SCF+RPE

TABLE VI
SENSITIVITY OF PA-DSE TO THE RPE CONFIDENCE THRESHOLD θ (BAMBU, B=120, n=40 PER CELL: 8 BENCHMARKS × 5 PERMUTATIONS). THE DEFAULT θ=0.8 USED THROUGHOUT THE PAPER IS IN BOLD. SR VARIES BY 18.8 PP ACROSS THE SWEPT RANGE. THE 0.1 PP DIFFERENCE BETWEEN THIS TABLE'S θ=0.8 ROW (83.1%) AND TABLE VII'S PA-DSE ROW (83.0%) REFLECTS THE DIFFERENT SAMPLE SIZES (n=40 VS. n=80).

θ	SR (%)	Wasted	TTFF (s)	Sig. activated
0.7	88.1 ± 0.7	7.6	9.3	1.0
0.8 (default)	$\mathbf{83.1 \pm 0.6}$	**11.4**	**9.3**	**1.0**
0.9	69.3 ± 0.4	24.8	9.3	1.0

row) costs 26.7 pp on Bambu. (iii) The middle tier (RPE's hard skip on typed recurrent evidence) is invisible at B=30–60 because OFRS surfaces feasible configurations before sufficient type-specific evidence accumulates, but becomes load-bearing at larger budgets (§V-I). (iv) Granting OFRS skip authority (SCF+OFRS-as-skip) breaks the evidence–action coupling and produces an order-of-magnitude increase in variance. Each tier in the hierarchy is the minimal action class supported by a qualitatively distinct evidence type, and removing or merging tiers degrades the policy along a predictable axis.

H. Sensitivity to RPE Confidence Threshold θ

The RPE activation threshold θ (§III-E1) governs how much concentrated evidence is required before a failure signature triggers hard skip. A lower θ activates signatures earlier (more aggressive pruning); a higher θ requires more evidence (more conservative). To probe the sensitivity of PA-DSE to this hyperparameter we sweep $\theta \in \{0.7, 0.8, 0.9\}$ on Bambu, all 8 benchmarks, 5 queue permutations per cell (n=40 runs per θ), with all other hyperparameters at their defaults (§IV-D).

Budget choice. At the interactive budgets used in the main evaluation (B=30 in Section V-G, B=60 in Section V-A), RPE rarely activates within the full PA-DSE configuration: OFRS reordering surfaces feasible configurations before enough same-type failures accumulate to trigger signature extraction. A meaningful sensitivity analysis must therefore be conducted at a budget where RPE actively contributes. We use B=120, twice the default Bambu budget, where RPE activation is consistently observed across all runs. Note that SR at B=120 is lower than at B=60 (Table III) because the denominator doubles: the queue now extends into higher-risk regions that OFRS defers at B=60. The per-evaluation success *rate* decreases because the marginal evaluations are riskier, even though the absolute count of feasible designs remains comparable ($\sim$ 56 at B=120 versus $\sim$ 55 at B=60 for PA-DSE).

Findings. Table VI summarises the sweep. Three observations stand out. First, θ is a meaningful lever: SR varies by 18.8 pp across the swept range (88.1% at θ=0.7 down to 69.3% at θ=0.9), and Wasted varies by 3× (7.6 → 24.8). Variance within each θ cell is small ($\sigma \leq 0.7$ pp), so the trend is statistically clean and not a sampling artifact. Second, the trend is driven by activation timing rather than activation count: across all 120 runs the number of activated signatures is consistently exactly 1. At θ=0.7, activation requires as few as 5 same-type failures with zero counterexamples ($5/(5+0+2) \approx 0.71 \geq 0.7$), so the signature triggers early in the run and most of the remaining budget benefits from the hard skip. At θ=0.8 the requirement rises to 8 failures ($8/10 = 0.8$), and at θ=0.9 to 18 ($18/20 = 0.9$), which often occurs only near the end of the budget, yielding little benefit—this is the core mechanism the evidence-hierarchy formulation predicts. Third, the default θ=0.8 is a balanced choice: it sits midway between the aggressive 0.7 and the conservative 0.9 regimes, accepting some loss in SR relative to 0.7 in exchange for tighter activation evidence ($\geq$ 8 failures versus 5 at θ=0.7). For a deployment that prioritises raw throughput at large budgets, θ=0.7 is preferable; for deployments where false skips are costly (e.g. when probe auditing is disabled), θ=0.9 may be safer despite the SR cost.

An identical sweep at B=60 provides a direct cross-check on the budget-dependence picture of §V-G. Across all three $\theta \in \{0.7, 0.8, 0.9\}$ (8 benchmarks × 5 permutations, n=40 per cell), the mean SR is 92.32%, mean wasted calls is 4.60, and signatures activated and probes triggered are both exactly 0 in every run; θ has *no* measurable effect at this budget, because RPE never reaches its activation condition within the full PA-DSE configuration. Raw logs for this B=60 sweep are included in the released repository (`results/theta_sweep/run_summary_fixed.csv`, one row per run with SR, wasted, signatures-activated, and probe counts). Sensitivity to θ is therefore conditional on RPE being in scope, which is itself conditional on budget being large enough for type-specific failure evidence to accumulate in the queue's reachable prefix.

I. RPE Contribution at Extended Budget

Framing. This subsection verifies that RPE—the middle tier of the evidence hierarchy (Table II)—is functional and load-bearing at budgets where its activation condition is met. It is not a claim that PA-DSE achieves higher SR at B=120 than at B=60: as noted in §V-H, the absolute SR is lower at B=120 (83.0% vs. 92.5%) because the queue extends into higher-risk regions that OFRS defers at B=60, increasing the denominator faster than the numerator. The relevant comparison at B=120 is PA-DSE versus SCF+OFRS at the same budget, which isolates RPE's contribution under conditions where it actually engages.

The ablation in Section V-G showed that at B=30 and B=60, RPE does not activate *in the full PA-DSE configuration* because OFRS reordering exhausts the budget before sufficient type-specific failure evidence accumulates; the SCF+RPE ablation row (Table V, 57.8% SR on Bambu at B=30) confirms that RPE itself is functional and the dormancy is a property of how OFRS interacts with the budget. To verify that RPE contributes *within the full configuration* at larger budgets, we run all methods at B=120 on Bambu (Table VII), adding SCF+OFRS (PA-DSE without RPE) as an explicit ablation control.

Figure 9 visualizes the B=120 result. At B=120, RPE activates exactly one signature per run (a recurrent pipeline-related failure pattern matching `pipeline=True`). This signature

TABLE VII

BAMBU AT B=120 (n=80 RUNS PER METHOD: 8 BENCHMARKS × 10 SEEDS/PERMUTATIONS). SCF+OFRS IS PA-DSE WITH RPE DISABLED. THE +36.3 PP GAP BETWEEN PA-DSE AND SCF+OFRS ISOLATES RPE'S CONTRIBUTION ($p < 0.001$, PAIRED t-TEST). RF AND SCF+OFRS EXHIBIT σ=0.0 BECAUSE AT B=120 BOTH METHODS HAVE OBSERVED ENOUGH CONFIGURATIONS TO FULLY RANK-ORDER THE ACTIVE SET, LEAVING NO SEED- OR PERMUTATION-INDUCED VARIATION. PA-DSE'S σ=0.6 REFLECTS SMALL VARIATION IN THE NUMBER OF CONSUMED CALLS ACROSS QUEUE PERMUTATIONS (TYPICALLY 67–68 OF 120); ALL PROBES FAIL IN THESE RUNS.

Method	SR (%)	Wasted	UQoR	Sigs
Random	13.7 ±2.1	103.6	9.8	–
Filtered Random	19.8 ±1.4	96.2	12.8	–
SA	7.2 ±14.7	111.3	3.2	–
GA	31.8 ±14.0	81.8	13.9	–
GP-BO	43.2 ±0.5	68.2	16.7	–
RF	46.7 ±0.0	64.0	19.1	–
SCF+OFRS	46.7 ±0.0	64.0	19.1	0
PA-DSE	**83.0** ±0.6	**11.5**	**19.1**	**1.0**

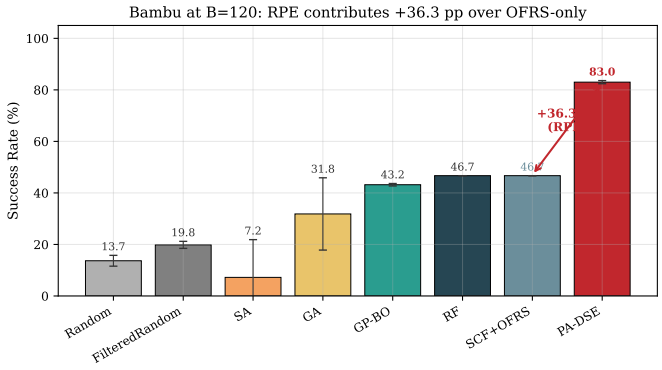

Fig. 9. Bambu at B=120. RPE activates one signature per run, hard-skipping ∼ 213 matching configurations and driving a +36.3 pp SR gain over SCF+OFRS (which is identical to RF at this budget).

initially covers the 224 pipeline-enabled configurations in the unblocked queue (the 420-point space contains 336 pipeline-enabled configurations; SCF removes 112 of these as blocked); because several matching configurations have already been evaluated before activation, RPE hard-skips on average ∼ 213 remaining queue entries that match the signature. This single signature drives PA-DSE from 46.7% (identical to SCF+OFRS and RF) to 83.0%, a +36.3 pp gain ($p < 0.001$, paired t-test over 80 benchmark-permutation pairs). The mechanism behind this gain deserves clarification: because SR is the fraction of evaluated configurations that synthesized successfully, RPE's hard-skipping shrinks the denominator. PA-DSE consumes approximately 68 of the 120 available calls and discovers ∼ 56 feasible designs; SCF+OFRS consumes the full 120 calls and also discovers ∼ 56. The two methods reach a comparable absolute count of feasible designs, but PA-DSE does so without spending the remaining ∼ 52 calls on the pipeline-conflict region. The +36.3 pp SR figure is therefore best read as a calls-to-success efficiency gain rather than a discovery gain, consistent with the contribution attributed to RPE in the evidence hierarchy. Probe auditing consumes approximately 1.5% of the budget actually consumed (an average of 1.0 probe per run); probes fail in every run, confirming the signature's validity across the full benchmark suite.

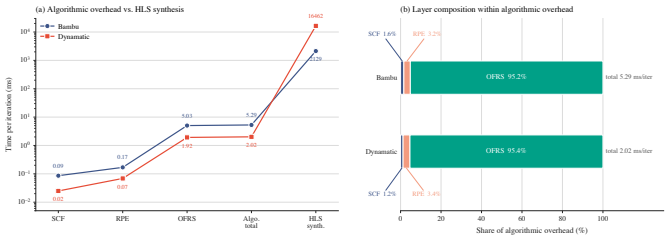

Fig. 10. Overhead analysis. (a) Per-iteration algorithmic cost is two-to-three orders of magnitude below synthesis cost on both tools. (b) Within the algorithmic overhead, OFRS dominates (it performs a queue sort each iteration); SCF and RPE are near-zero.

The mechanism is clear: at B=60, OFRS surfaces feasible configurations fast enough that the budget is exhausted before τ=2 same-type failures accumulate in the queue's reachable prefix. At B=120, the queue extends into the high-failure-rate pipelining region, RPE observes repeated pipeline-related failures, extracts a discriminative signature, and hard-skips matching configurations not yet evaluated (∼ 213 on average per run) for the remainder of the budget. OFRS alone cannot achieve this because its per-dimension marginal statistics do not isolate type-specific failure clusters. RPE's type-specific, concentrated evidence is what justifies the hard-skip action that OFRS's mixed-type marginal statistics do not support.

J. Overhead

Table VIII reports the per-iteration algorithmic overhead of each PA-DSE layer, compared to the corresponding synthesis cost; Figure 10 visualizes the same data and the per-layer breakdown within the algorithmic overhead. Synthesis times are the per-call mean from the exhaustive ground-truth evaluation. On Bambu, total PA-DSE overhead is 5.3 ms per iteration, versus ≈ 2129 ms of synthesis: a ratio of 1:403. On Dynamatic the ratio is 1:8164, reflecting the higher synthesis cost of MILP-based buffer placement. By either measure, algorithmic overhead is negligible.

OFRS dominates the algorithmic overhead (95% on Bambu, 95% on Dynamatic) because it performs a full-queue sort every iteration. SCF is essentially free (rules fire once at initialization), and RPE's overhead is concentrated in a small number of signature-extraction calls per run. For design spaces substantially larger than ours, the OFRS sort could be replaced by an incremental priority queue with $O(\log n)$ update; we have not needed this optimization for the scales evaluated here.

VI. DISCUSSION AND LIMITATIONS

A. When PA-DSE Helps, and When It Does Not

PA-DSE is most effective in the regime our evaluation targets: small-to-moderate exploration budgets on design spaces where a non-trivial fraction of configurations are infeasible. Outside this regime its advantages diminish, and we state the limits explicitly.

Very small budgets ($B < n_{\min}$). OFRS requires $n_{\min}$=5 observations per dimension before its relevance weights stabilize. Below this threshold, OFRS falls back to a passive

TABLE VIII
PER-ITERATION OVERHEAD DECOMPOSITION (MS). PA-DSE'S ALGORITHMIC COST IS $< 0.25\%$ OF SYNTHESIS COST ON BAMBU AND $< 0.02\%$ ON DYNAMATIC.

Tool	SCF	RPE	OFRS	Algo. total	Synthesis
Bambu	0.086	0.169	5.030	5.285	2129.0
Dynamatic	0.025	0.069	1.923	2.017	16462.2

mode that preserves the initial queue order. In the reported permutation-based experiments this order is randomized, so suppression remains only a risk label rather than an active-before-suppressed guarantee. In this regime PA-DSE behaves like Filtered Random over the unblocked space, with only the static filtering benefit remaining (140 documented incompatibilities removed). At $B{=}10$ we would expect the gap to RF to narrow.

Large budgets ($B \gg |\mathcal{C}_{\text{act}}|$). At budgets approaching the full active-set size, every method converges to the same coverage. The pertinent question becomes not SR but QoR quality, where learned surrogates with offline training can leverage more data. PA-DSE is not designed for this regime.

Tools without documented incompatibility rules. On Dynamatic, SCF reduces to a no-op and PA-DSE's SR lead over the best baseline narrows to $+1.1$ pp. The more distinctive advantage shifts to TTFF (34.2 s vs. 67.0 s, nearly $2\times$ faster) and UQoR (7.1 vs. GP-BO 5.1). DFRL alone is sufficient to match or exceed all baselines without tool-specific rule engineering. For tools whose failure distribution is driven by runtime-detected issues (e.g., timing closure, resource overflow), DFRL's online learning provides a viable fallback.

B. Quality–Reliability Tradeoff

Among methods with SR $\geq$ 85%, PA-DSE leads on reliability-oriented metrics (SR, Wasted, TTFF) and remains competitive on QoR (Table IV). The only methods with lower best latency (Random, GA) achieve SR below 65%, making them unsuitable for production use. We examine this tradeoff below.

Low-latency baselines pay for it in SR. Among the five non-trivial Dynamatic benchmarks, Random achieves lower best latency than PA-DSE but at catastrophic SR (49.9%); GA shows a similar pattern (64.5% SR). These methods sample the design space broadly, which occasionally discovers low-latency configurations that PA-DSE does not reach within the limited budget, but the same broad sampling produces their catastrophic Wasted counts (10–15 calls per 30-budget run). Figure 11 visualizes this trade-off: methods inside the shaded production-feasible region (SR $\geq$ 85%) cluster around the Pareto frontier; methods outside trade tens of percentage points of SR for marginal latency gain. For a production DSE policy, a method that wastes one third to one half of every run on infeasible configurations is not deployable, regardless of its QoR numbers.

Among deployable methods, PA-DSE compares favorably on user-visible cost. GP-BO and RF are the only baselines with SR $\geq$ 85%. PA-DSE's SR is comparable to GP-BO (91.3% vs. 90.2%) while halving TTFF (34.2 s vs. 67.0 s) and reducing Wasted (2.6 vs. 2.9). UQoR is notably higher (7.1 vs. 5.1). Best area is matched (51.3 vs. 51.3). Best latency remains close to GP-BO (407.6 vs. 405.4, a 2.2-cycle / 0.5% gap). Against RF, the gap is wider: SR $+5.7$ pp, Wasted -1.7, TTFF -32.8 s.

Scope of comparison. The meaningful comparison is among deployable methods (SR $\geq$ 85%), where PA-DSE compares favorably on reliability-oriented metrics while remaining close on QoR. Random and GA report lower best latency, but at SR below 65%, which limits their practical utility in production workflows. We adopt SR $\geq$ 85% as a working deployment threshold: below this, more than one synthesis call in seven is wasted per run, which becomes a perceptible cost in interactive workflows where developers iterate tens of times a day. The qualitative conclusions are insensitive to the exact cutoff: at any threshold between 80% and 90%, Random, Filtered Random, SA, and GA fall outside the deployable set, while PA-DSE, GP-BO, and RF fall inside.

C. Decision Traceability

Every skip decision in PA-DSE is attributable to either (a) a named static rule or (b) a named RPE signature with a known support count and confidence. At the end of a run, one can inspect `signatures_learned` and the associated `conditions` columns to see exactly which parameter combinations the algorithm treated as failure-prone. We use the term *decision traceability* rather than *interpretability*: the framework exposes why the policy took an action, but does not claim a causal explanation for the underlying tool's failure—a signature captures a statistical conjunction, not a root cause. This remains a pragmatic advantage over surrogate-based methods, where a confidently-predicted infeasibility is not attributable to a named piece of state at all.

D. Generalization Across Runs

PA-DSE's DFRL state does not persist across runs. Each invocation starts with empty RPE signatures and empty OFRS statistics, re-learning within the budget. On tools like Dynamatic where failure modes are graph-complexity-driven, failure signatures learned on one benchmark may not transfer to another, and persisting them risks false skips. Cross-run persistence for the *same* benchmark is a natural extension: hashing the source AST and caching previously-learned signatures would yield a warm-start variant. We have not implemented this.

E. Threats to Validity

Tool versions. Our evaluation uses Bambu 0.9.8 and Dynamatic 2.0 with Gurobi 12.0. Different tool versions may expose

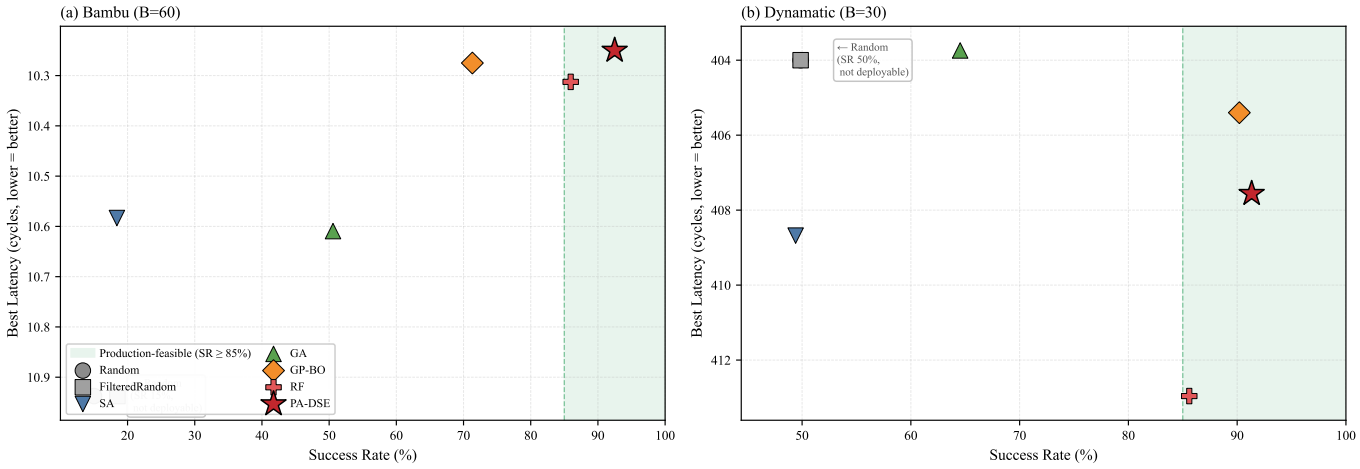

Fig. 11. Reliability–quality Pareto trade-off: Success Rate (x-axis) vs. best latency (y-axis, lower is better). Shaded regions mark the production-feasible operating regime (SR $\geq 85\%$). On both Bambu (a) and Dynamatic (b), PA-DSE attains the Pareto frontier within the feasible region. On Dynamatic, methods outside the shaded region (Random, FilteredRandom, SA, GA) achieve lower best latency only by sacrificing tens of percentage points of SR, making them non-deployable as production policies regardless of their QoR numbers.

different failure modes; the RPE signatures learned for Bambu 0.9.8 are not guaranteed to transfer to future releases.

Benchmark suite. Our eight benchmarks (shared across both tools) are small-to-medium in size ($<$ 1000 lines of C each). For kernel-style benchmarks this is representative; for full-application HLS, longer compile times would shift the overhead ratio further in PA-DSE's favor, but the feasibility landscape may differ qualitatively.

Permutation-based repetition. For PA-DSE we report 10 queue permutations rather than 10 seeds. The two are mechanically equivalent for a deterministic algorithm (permutation fixes the initial queue order, a seed fixes internal randomness in a stochastic algorithm), but readers should note this is not a statistical sample of PA-DSE's stochasticity per se; PA-DSE is deterministic conditional on the queue order. The p_{probe}=0.05 introduces residual stochasticity at extended budgets where RPE activates (cf. Table VII, σ=0.6 at B=120, driven by small variation in consumed-call count across permutations); at B=60 on Bambu, RPE does not activate and no probes are triggered, so the observed σ=1.1 reflects across-permutation variation alone.

RF baseline fidelity. Our RF baseline is a simplified online proxy for the AutoScaleDSE family; a faithful re-implementation would require offline corpus training across multiple HLS designs and tool versions. The RF numbers in this paper should be read as representative of what an online random-forest classifier achieves under our conditions, not as an upper bound on the learned-surrogate family.

No direct comparison against recent surrogate methods. We did not retrain recent GNN-based feasibility predictors (e.g., HGBO-DSE [10], IronMan [8]) on our configuration spaces, as doing so would require retraining on our specific tool versions and pragma ranges. The RF baseline serves as a proxy for the learned-classifier family under our evaluation conditions. A direct comparison with offline-trained GNN surrogates would further contextualize PA-DSE's relative strengths and is planned as future work.

VII. Conclusion

We presented PA-DSE, a feasibility-aware HLS design space exploration policy built around the principle that action strength must not exceed evidence strength. The policy has two layers: a static constraint filter (SCF) that permanently removes configurations matching tool-documented incompatibility rules, and a dynamic failure risk learner (DFRL) that accumulates evidence during a single exploration run and reorders the queue accordingly. DFRL has two sub-components, recurrent pattern extraction (RPE) for signature-based hard skipping and online failure risk scoring (OFRS) for continuous queue reordering, each restricted to actions its evidence can justify.

On eight Bambu benchmarks at B=60, PA-DSE achieves 92.5% success rate with variance matching the lowest among non-trivial methods (tied with RF at σ=1.1), outperforming a random-forest baseline by +6.6 pp and reducing wasted synthesis calls by 1.9×. On the same eight benchmarks evaluated on Dynamatic at B=30, where no applicable static rules exist and SCF reduces to a no-op, PA-DSE reaches 91.3% success rate on the five benchmarks with non-trivial failure rates and halves the time-to-first-feasible of the strongest baselines, with QoR closely matching deployable baselines (matched best area, full Pareto-vertex coverage, and best latency within 0.6% of GP-BO). Three benchmarks achieve 100% feasibility on Dynamatic but not on Bambu, a structural asymmetry that motivates the two-layer design: SCF handles pragma-driven failures, DFRL handles graph-complexity-driven failures. The 8-way ablation matches each component's engagement to its predicted position in the evidence hierarchy: at the interactive budgets of our main evaluation (B=30–60), OFRS's weak-evidence online reordering is the dominant within-budget contributor, while RPE remains dormant *within the full PA-DSE configuration* because OFRS exhausts the budget before sufficient type-specific failure evidence accumulates (the SCF+RPE ablation row, in which OFRS is disabled, confirms RPE itself is functional); only at the extended budget B=120

does RPE activate within the full configuration and add $+36.3$ pp over OFRS-only through hard-skipping configurations matching a learned pipeline-related failure signature. SCF's strong-evidence permanent block serves as a safety net on tools that expose one, and allowing OFRS to hard-skip causes variance to grow by an order of magnitude. A sensitivity sweep of the RPE confidence threshold θ at $B{=}120$ shows that activation timing rather than signature count drives performance, with the default $\theta{=}0.8$ sitting between aggressive early activation ($\theta{=}0.7$, SR 88%) and conservative late activation ($\theta{=}0.9$, SR 69%). The total algorithmic overhead is 5.3 ms per iteration on Bambu and 2.0 ms on Dynamatic, negligible compared to the 2.1 s and 16.5 s per-call synthesis costs.

The framework requires no offline training and operates within a single exploration run; every skip decision is attributable to a named rule or signature with a known support count. We do not claim causal explanation of the tool's underlying behaviour. The principal contribution is a design pattern—couple action strength to evidence strength—rather than a specific set of rules or scoring functions. Promising extensions include QoR-aware queue reordering (e.g., novelty bonuses or density penalties) for settings where the success cloud is narrow, cross-run signature caching for repeated benchmark exploration, higher-order interaction-aware risk scoring for spaces where single-dimension statistics saturate, and integration with a probe-rate controller that actively modulates the quality–reliability tradeoff.

Code and experimental data are released at https://github.com/jane09250410/hls-dse.

References

[1] H. Jun, H. Ye, H. Jeong, and D. Chen, "AutoScaleDSE: A scalable design space exploration engine for high-level synthesis," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 16, no. 3, 2023.

[2] L. Ferretti, G. Ansaloni, and L. Pozzi, "Lattice-traversing design space exploration for high-level synthesis," in *Proceedings of the 36th IEEE International Conference on Computer Design (ICCD)*. IEEE, 2018, pp. 210–217.

[3] C. Pilato and F. Ferrandi, "Bambu: A modular framework for the high level synthesis of memory-intensive applications," in *Proceedings of the 23rd International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2013, pp. 1–4.

[4] L. Josipović, R. Ghosal, and P. Ienne, "Dynamically scheduled high-level synthesis," in *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. ACM, 2018, pp. 127–136.

[5] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems 25 (NeurIPS)*, 2012, pp. 2951–2959.

[6] L. Ferretti, G. Ansaloni, and L. Pozzi, "Cluster-based heuristic for high level synthesis design space exploration," *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 1, pp. 35–43, 2021.

[7] H.-Y. Liu and L. P. Carloni, "On learning-based methods for design-space exploration with high-level synthesis," in *Proceedings of the 50th Annual Design Automation Conference (DAC)*. ACM, 2013, pp. 1–7.

[8] N. Wu, Y. Xie, and C. Hao, "IronMan: GNN-assisted design space exploration in high-level synthesis via reinforcement learning," in *Proceedings of the 2021 Great Lakes Symposium on VLSI (GLSVLSI)*. ACM, 2021, pp. 39–44.

[9] A. Sohrabizadeh, Y. Bai, Y. Sun, and J. Cong, "Automated accelerator optimization aided by graph neural networks," in *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*. ACM, 2022, pp. 55–60.

[10] H. Kuang, X. Cao, J. Li, and L. Wang, "HGBO-DSE: Hierarchical GNN and Bayesian optimization based HLS design space exploration," in *Proceedings of the 2023 International Conference on Field Programmable Technology (ICFPT)*. Yokohama, Japan: IEEE, 2023, pp. 106–114.

[11] M. R. Ahmed, T. Koike-Akino, K. Parsons, and Y. Wang, "AutoHLS: Learning to accelerate design space exploration for HLS designs," in *Proceedings of the 2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)*. IEEE, 2023, pp. 491–495.

[12] E. Ustun, C. Deng, D. Pal, Z. Li, and Z. Zhang, "Accurate operation delay prediction for FPGA HLS using graph neural networks," in *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*. ACM, 2020, pp. 1–9.

[13] E. Murphy and L. Josipović, "Balor: HLS source code evaluator based on custom graphs and hierarchical GNNs," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. Newark, NJ, USA: ACM, 2024.